FH University of Applied Sciences TECHNIKUM WIEN

MASTER THESIS

Thesis submitted in fulfillment of the requirements for the degree of Master of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program Embedded Systems

C and Rust in Embedded Real-Time Systems

A Controlled Study of Performance, Memory Usage, and Safety Boundaries

By: Simon Nenning BSc.

Student Number: 2410297031

Supervisor 1: FH-Prof. Dipl.-Ing. Dr. Martin Horauer

Supervisor 2: Volker Tenta, MSc.

Wien, 2026-10-21

Declaration of Authenticity

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz/ Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and according to the rules currently applicable at the UAS Technikum Wien and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.”

Wien, 2026-10-21 *place your digital signature here*

Kurzfassung

Diese Masterarbeit vergleicht die Programmiersprachen C und Rust beim Einsatz auf eingebetteten Echtzeitbetriebssystemen. Untersucht werden Ausführungszeit, Speicherbedarf, Speichersicherheit und Implementierungskomplexität. Der Vergleich umfasst vier Hardwareplattformen sowie die Betriebssysteme FreeRTOS, NuttX und Zephyr, auf denen dieselben Testfälle in C und Rust umgesetzt wurden. Zusätzlich werden die Rust-Kernel FreeRTOS-in-Rust und Rivet betrachtet.

Für den Vergleich wurde ein gemeinsames Testverfahren mit 26 Testfällen entwickelt, die Task-Verwaltung, Synchronisation, Task-Kommunikation und Interruptverarbeitung untersuchen. Insgesamt wurden 27 Kombinationen aus Hardware, Betriebssystem und Programmiersprache auf den jeweiligen Geräten in zehn unabhängigen Messläufen getestet. Neben den Laufzeiten wurden auch der Speicherbedarf, sicherheitsrelevante Operationen im Quellcode und die Komplexität der Implementierungen untersucht.

Die Messungen zeigen, dass C und Rust auf derselben Hardware und mit demselben Betriebssystem in den meisten Testfällen ähnlich schnell sind. Deutlichere Unterschiede treten vor allem bei der Kommunikation zwischen Tasks auf. Auch beim Speicherbedarf gibt es Unterschiede: Während der zusätzliche Arbeitsspeicherbedarf pro Task identisch ist, benötigen die Rust-Implementierungen meist mehr Flash-Speicher und teilweise auch mehr statischen Arbeitsspeicher.

Im untersuchten Rust-Anwendungscode wurde keine direkte Rohzeigeroperation eindeutig aufgelöst; je NuttX-Konfiguration blieb ein Eintrag ungeklärt. Im entsprechenden C-Code kommen solche Operationen dagegen vor. Sobald Rust mit einem in C geschriebenen Betriebssystemkern zusammenarbeitet, sind jedoch weiterhin Operationen notwendig, deren Sicherheit der Rust-Compiler nicht überprüfen kann. Die Ergebnisse zeigen somit, wo solche Operationen auftreten, erlauben aber keine Aussage darüber, ob ein vollständiges System speichersicher ist. Darüber hinaus benötigen die Rust-Implementierungen weniger selbst geschriebenen Code, erfordern aber zusätzliche Schnittstellen zum Betriebssystem.

Rust erreicht in den untersuchten Konfigurationen eine mit C vergleichbare Laufzeitleistung; bei der Sprachwahl sind Speicherbedarf, Schnittstellen und Wartungsaufwand zu berücksichtigen. Für die übernommenen Ergebnisse der Rust-basierten Kernel fehlen die Rohdaten.

Schlagwörter: Eingebettete Systeme, Echtzeitbetriebssysteme, Programmiersprachenvergleich, Rust, Speicherbedarf, Speichersicherheit, Laufzeitverhalten, Reproduzierbarkeit

Abstract

This thesis compares C and Rust applications on embedded real-time operating systems, with the aim of distinguishing the effects of the application language from those of the operating system kernel and hardware. The evaluation covers four hardware platforms and five kernel implementations. FreeRTOS, NuttX and Zephyr are examined with applications written in both languages, while the Rust-native systems FreeRTOS-in-Rust and Rivet are evaluated separately.

A common benchmarking framework was developed with 26 test cases covering scheduling, synchronisation, communication, interrupt handling and other operating system operations. In total, 27 firmware configurations were evaluated on physical hardware, with ten independent measurement runs per configuration. Execution times were examined alongside memory consumption, safety-relevant source operations and integration complexity. The measurements and corresponding build configurations were archived to support reproducibility.

The results show that C and Rust achieve similar execution times for most benchmark operations when the hardware and kernel remain unchanged. Larger differences occur mainly in task communication and depend on the platform and implementation. Per-task memory overhead is identical between matched C and Rust configurations, while Rust-linked firmware generally requires more flash memory and sometimes more static random-access memory.

The safety analysis finds no resolved, directly expressed raw-pointer operations in the analysed Rust application code, whereas such operations occur in the corresponding C applications. However, raw-pointer operations remain present in the interfaces connecting Rust applications to C-based kernels. These findings do not establish complete-firmware memory safety. The Rust implementations also contain fewer lines of hand-written application and boundary code, although their kernel interfaces introduce additional maintenance requirements.

Overall, Rust achieves performance comparable to C for the investigated workloads, but memory requirements, integration boundaries and maintenance implications must be considered for each target system. The archive supports further comparisons; raw data for the retained Rust-native summaries are unavailable.

Keywords: real-time operating systems, embedded systems, programming language comparison, Rust, memory consumption, memory safety, performance evaluation, reproducibility

Acknowledgement

I would like to thank my supervisor, Dr. Martin Horauer, for his patience, guidance and willingness to help throughout the development of this thesis. I particularly appreciate his understanding and continued support, even during periods when my communication was less frequent than it should have been.

I would also like to thank my friends and family for their encouragement, patience and support throughout my studies. I am especially grateful to those who took the time to proofread this thesis and provide valuable feedback.

Table of Contents

1 Introduction 8
2 Background 10
2.1 Task Timing Concepts 10
2.2 Task Scheduling and Priorities 11
2.3 Event Handling and Task Communication 13
2.4 Memory and Resource Accounting 15
2.5 Memory Safety and Interoperability 15
2.6 Software Complexity and Maintainability 17
3 State of the Art 18
3.1 RTOS Benchmarking Approaches 18
3.2 Comparisons of Real-Time Operating Systems 19
3.3 C and Rust in Embedded Systems 20
3.4 Rust Applications on C-Based Kernels 21
4 Experimental Setup 24
4.1 Hardware Platforms 24
4.2 Timing Sources 25
4.3 RTOS Configurations 25
4.4 Benchmarking Framework 26
4.5 Build Environment 26
4.6 Communication Protocol 27
5 Methodology 28
5.1 Benchmark Suite 28
5.2 Measurement Procedure 30
5.3 Statistical Evaluation 30
5.4 Resource Measurement 31
5.5 Safety Analysis 32
5.6 Complexity and Maintainability 33
6 Results 35
6.1 Benchmark Measurements 35
6.2 Memory Usage 38
6.3 Static Safety Analysis 39
6.4 Integration Complexity 42
7 Discussion 44
7.1 Performance and Resources 44
7.2 Safety at the Boundary 45

7.3 Integration Complexity 46
8 Conclusion 47
8.1 Limitations 48
8.2 Future Work 48
Bibliography 49
List of Abbreviations 55
Documentation of AI Assistance 56
A Benchmark Specifications 57
B Complete Benchmark Results 62
C Complete Memory Results 74
D Complete Safety Analysis Results 76
E Complete Integration and Complexity Results 78

1 Introduction

Systems programming languages such as C and C++ provide direct control over memory and hardware, making them widely used in operating systems, firmware, and performance-critical software. However, they offer limited language-level protection against memory-safety errors, including buffer overflows, use-after-free, and dangling pointers. Such defects are a significant source of software vulnerabilities and remain a longstanding concern in systems programming [1].

The importance of memory safety has also gained recognition at the policy level. In February 2024, the United States Office of the National Cyber Director published Back to the Building Blocks, advocating the adoption of memory-safe programming languages as a strategic measure to reduce software vulnerabilities [2].

Memory-safety errors are especially dangerous in embedded systems, where hardware protection is limited. Many microcontrollers have no memory-management unit. Protection then depends on whether a memory-protection unit is present and correctly configured [3]. Once deployed, such a device becomes hard to patch, and field studies show that such flaws are widespread in consumer and industrial firmware [4].

The stakes are rising as well. The defects that survive in the field are a standing target, and the tools for finding them keep improving. Machine learning and large language models are increasingly used for such vulnerability detection [5]. Once a flaw is found, it can then be turned against the device. An attacker can take over the device, corrupt or extract its data, or disrupt its operation. Limited update capabilities can leave vulnerabilities exploitable for years, potentially causing operational and financial harm.

Rust aims to address these memory-safety problems while retaining the low-level control required for systems programming. Rust uses ownership, borrowing, and its type system to prevent broad classes of memory-safety errors and data races at compile time [6]. It achieves this without requiring a garbage collector [7].

However, embedded development often requires operations that Rust's compiler cannot verify. These include direct hardware access and interactions with C-based real-time operating system (RTOS) kernels. Rust allows such operations to be encapsulated in wrappers, limiting the amount of unsafe code. Nevertheless, these wrappers do not eliminate the underlying risks or make the surrounding C code safe. Such interactions introduce additional integration requirements and potential performance overhead, both of which are relevant when comparing C and Rust in embedded RTOS environments.

These challenges complicate comparisons between C and Rust in embedded RTOS environments. Observed differences may stem not only from the programming language but also from the RTOS kernel and the mechanisms used for C/Rust interoperability. Previous research has also identified challenges in tooling and interoperability when combining both languages [8]. To account for these factors, this thesis aims to isolate their influence by varying individual components while keeping the remaining experimental conditions constant.

The comparison covers four hardware platforms and five RTOS kernel implementations, including C-based and Rust-based kernels. Equivalent workloads are implemented in C and Rust to measure execution time and memory consumption. Memory safety and integration complexity are examined separately.

The investigation addresses four objectives:

1. **Performance and resource consumption.** The first objective is to examine differences in execution time and memory consumption between equivalent C and Rust implementations. For direct language comparisons, the hardware platform and RTOS kernel are kept constant.
2. **Memory safety.** The second objective is to investigate the memory-safety characteristics of both languages in embedded RTOS environments. Particular attention is given to unsafe operations and interactions between Rust applications and C-based kernels.
3. **Maintainability and integration.** The third objective is to examine the complexity of implementing and maintaining equivalent workloads in both languages. This includes the required adapters, build configurations, and platform-specific code.
4. **Validity and reproducibility.** The final objective is to establish which experimental conditions must be controlled and documented to support meaningful and reproducible comparisons. This includes identifying the influence of the programming language, RTOS kernel, and integration mechanisms.

The evaluation is limited to the selected hardware platforms, RTOS implementations, and benchmark workloads. Power consumption is not investigated, and the measured execution times do not establish worst-case execution-time guarantees. Similarly, the memory-safety analysis does not establish the safety of the complete system.

These objectives do not presume that either language is superior in every category.

2 Background

Comparing C and Rust on embedded real-time operating systems (RTOS) requires an understanding of how these systems operate and how their performance is measured. Important concepts include task scheduling, execution time, response time, and deadlines.

This chapter introduces these concepts and explains the role of memory management, programming language characteristics, and C/Rust interoperability in embedded systems. It provides the technical foundation for the experimental methodology in the Methodology chapter.

2.1 Task Timing Concepts

In real-time systems, correct behaviour depends not only on producing the expected result but also on producing it within a required time frame. A task may perform its computation correctly and still fail if it finishes too late. Understanding execution time, response time, and deadlines is therefore essential when evaluating RTOS performance.

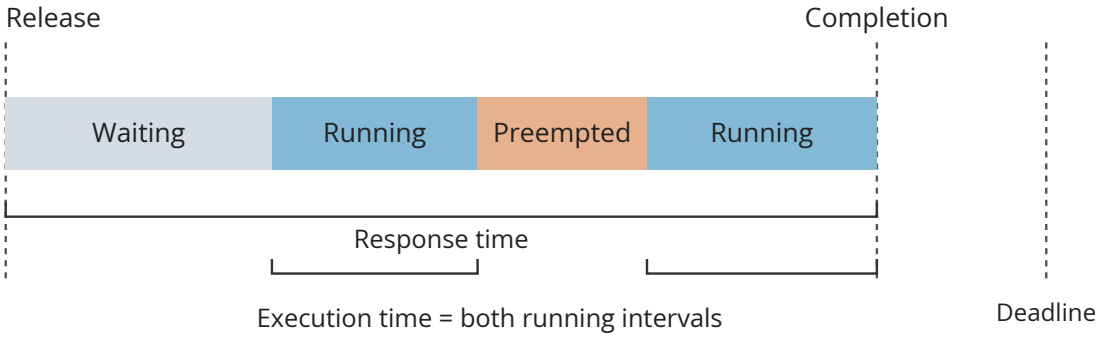


Fig. 1: Task timing relationships

Fig. 1 illustrates a task's execution from release to deadline. After release, the task initially waits for the processor, shown in grey. It then executes, shown in blue, before being preempted, shown in orange. Once it resumes, it continues executing until completion.

These intervals are used to distinguish two timing measurements [9]:

- **Execution time:** The total time a task spends running on the processor, shown by the two blue intervals.
- **Response time:** The time from task release to completion, including waiting and preemption.

The difference matters because a task may require little processor time but still finish too late due to scheduling delays. In Fig. 1, the task completes before its deadline. The remaining time is called the **deadline margin**. Additional delays would reduce this margin and could eventually cause a deadline miss.

However, the diagram represents only one execution. Measured response times do not establish a worst-case bound or guarantee that future deadlines will be met.

> **Key takeaway**
>
> Execution time measures processor usage, while response time includes waiting and preemption. Distinguishing them helps interpret RTOS timing measurements, but observed times do not guarantee future deadlines.

2.2 Task Scheduling and Priorities

Task response time includes periods when a task is waiting rather than executing. These delays depend partly on how the RTOS schedules tasks and manages their priorities.

In **preemptive scheduling**, a higher-priority task can interrupt a running lower-priority task when it becomes ready. In **cooperative scheduling**, the running task continues until it yields or blocks, even if a higher-priority task is ready to execute [10].

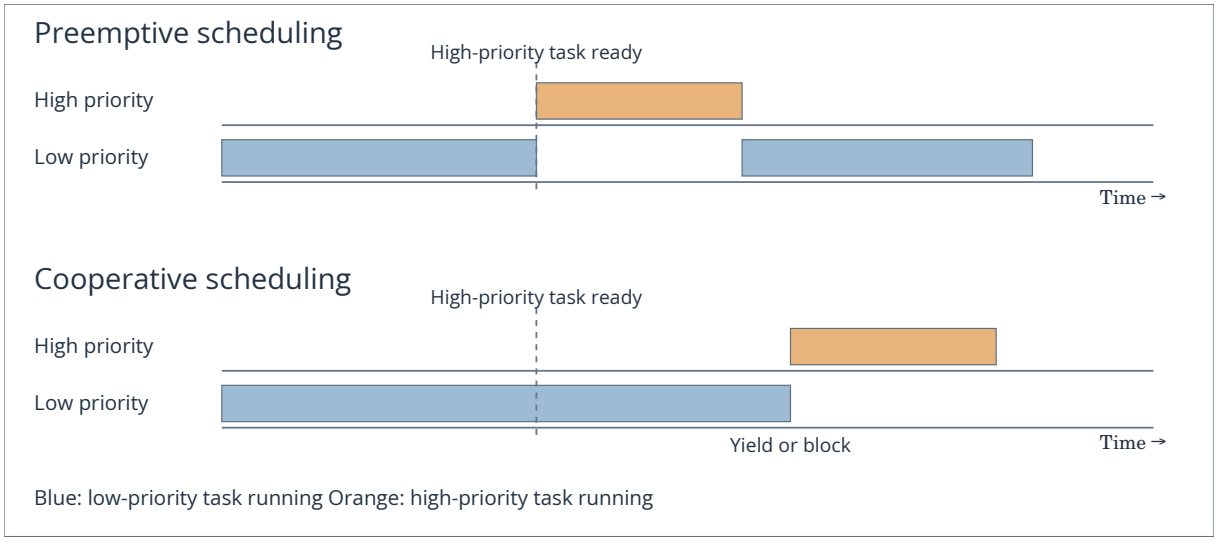


Fig. 2: Preemptive and cooperative scheduling

Fig. 2 compares both scheduling modes using the same task scenario. The dashed line marks when the high-priority task becomes ready. Under cooperative scheduling, its execution is delayed until the running task yields or blocks.

Scheduling also affects tasks that wait for a specified amount of time. How precisely the kernel handles these delays is called **scheduling granularity**.

One way an RTOS manages these delays is through a periodic **system tick**. This is a timer interrupt that occurs at fixed intervals. The kernel uses each tick to update its timing information and check whether tasks should wake up. For example, with a system tick of 1 ms, a requested delay of 0.5 ms may be rounded up to a full tick. The task then waits longer than requested because of insufficient scheduling granularity. This increases its response time without increasing the time it spends executing on the processor.

While scheduling granularity affects when a task can resume, **measurement resolution** determines how precisely its timing can be measured. It describes the smallest time interval a timer can distinguish [10].

For example, a hardware counter may measure time in microseconds even when the kernel schedules delays in milliseconds. This allows both execution time and response time to be measured more precisely, but does not reduce the task's waiting time. Sufficient measurement resolution is therefore important for distinguishing actual execution costs from delays introduced by the RTOS.

Waiting times can also arise when tasks compete for shared resources. A **mutex** protects a shared resource by allowing only one task to hold it at a time. If another task attempts to acquire the same mutex using a blocking operation, it must wait until the current holder releases it.

This can lead to **priority inversion**. For example, a low-priority task may acquire a mutex before a high-priority task needs it. The high-priority task then blocks until the mutex is released. However, if a medium-priority task becomes ready, it can preempt the low-priority task. This prevents the mutex holder from continuing its work and extends the waiting time of the high-priority task.

Priority inheritance addresses this problem by temporarily raising the mutex holder's priority to that of the waiting high-priority task. This prevents medium-priority tasks from preempting the mutex holder while it completes its critical section. Once the mutex is released, the holder's priority is restored, and the high-priority task can proceed [11].

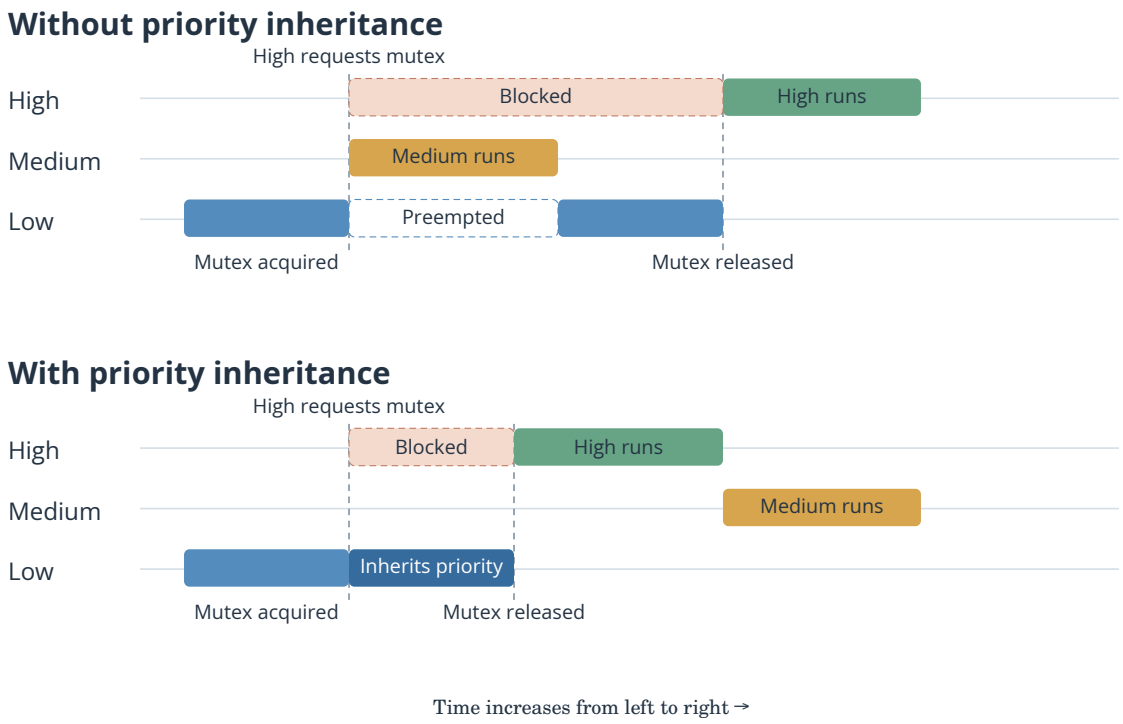


Fig. 3: Priority inversion and inheritance

Fig. 3 illustrates how priority inheritance changes task execution and waiting time.

> **Key takeaway**
>
> Scheduling policies, timer granularity, and resource contention influence how long tasks wait. These effects can change benchmark timings independently of whether the application is written in C or Rust.

2.3 Event Handling and Task Communication

Task execution is influenced not only by scheduling decisions, but also by events that occur outside the running task. Such events may require immediate attention or cause a waiting task to become ready.

Interrupts allow an RTOS to respond to external events without requiring a task to continuously check for them. When an interrupt occurs, the processor temporarily suspends its current execution and runs an **interrupt service routine** (ISR) [9].

The ISR handles time-sensitive work, while more complex processing can be deferred to a regular task. For example, an ISR may signal that new sensor data is available. A task waiting for this event then becomes ready to execute. However, it may still need to wait until the scheduler allows it to run.

Interrupt handling therefore involves two distinct delays. The processor must first respond to the interrupt, and the waiting task must then be scheduled to run.

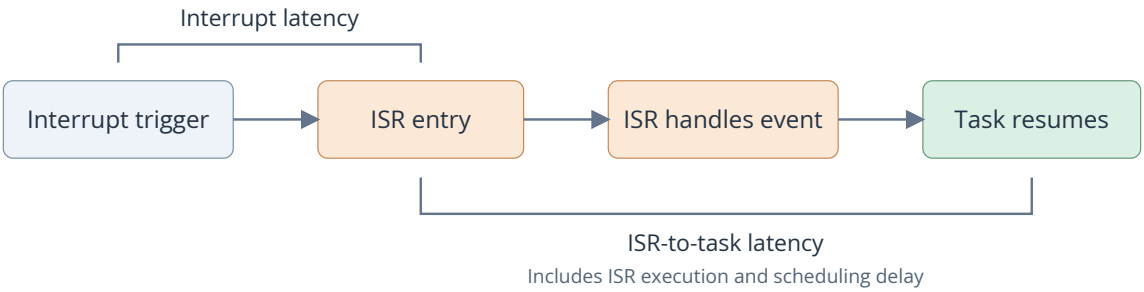


Fig. 4: Interrupt handling sequence

Fig. 4 illustrates these stages and distinguishes two timing measurements:

- **Interrupt latency:** The time from the interrupt trigger to ISR entry. It includes delays caused by interrupt masking, higher-priority interrupts, and the processor's initial interrupt handling.
- **ISR-to-task latency:** The time from ISR entry until the waiting task resumes execution. It includes ISR execution, task notification, and any scheduling delay before the task runs.

Interrupt handling can also be affected by other system activity. For example, interrupts may be temporarily disabled during critical sections, delaying the execution of an ISR [10]. Higher-priority interrupts may also interrupt an ISR that is already running. This is known as **nested interrupt handling** [9]. Both situations can introduce additional delays and affect the measured interrupt response time.

Beyond handling the interrupt itself, the RTOS must often coordinate further processing between tasks. This involves **synchronisation** and communication mechanisms. Message queues transfer information between tasks, semaphores can signal events, and mutexes protect shared resources.

These operations may also introduce delays. For example, sending a message through a queue may require copying its contents [12]. If the queue is full, the sending task may have to wait until space becomes available [10]. The resulting latency therefore depends on factors such as payload size and blocking behaviour.

> **Key takeaway**
>
> Event handling and task communication introduce delays at different stages. Interrupt handling, scheduling, payload size, and blocking behaviour must be considered when interpreting RTOS latency measurements.

2.4 Memory and Resource Accounting

Besides timing behaviour, memory consumption is an important consideration in embedded systems, where available flash and RAM are often limited. Memory requirements can be divided into **static footprint** and **runtime memory usage**.

The static footprint is determined from the compiled program. Code and constant data generally occupy flash, while initialised and zero-initialised data require RAM. Initial values for initialised data are typically stored in flash and copied to RAM during startup. The memory required for code and data can be examined using the sections of the Executable and Linkable Format (ELF) file.

Runtime memory use, by contrast, depends on program execution and includes stack consumption and heap allocations. Measuring peak stack and heap usage helps estimate the memory required under a particular workload.

Memory management can also introduce execution-time overhead. Each task requires kernel bookkeeping structures, while dynamic allocation takes time and may lead to heap fragmentation. Fragmentation occurs when free memory is divided into separate blocks, potentially preventing larger allocations even when sufficient total memory remains [13].

> **Key takeaway**
>
> Static footprint, runtime memory use, and allocation behaviour describe different resource costs. Comparing them alongside timing measurements gives a more complete picture of C and Rust implementations.

2.5 Memory Safety and Interoperability

Besides the amount of memory an application requires, it is also important how that memory is managed and accessed. This is a key difference between C and Rust.

In C, programmers must ensure that pointers remain valid, memory is managed correctly, and shared resources are accessed safely. Rust uses **ownership** and **borrowing** to enforce many of these rules at compile time. Each value has an owner, while the borrow checker restricts how references can be used and how long they remain valid [7].

These differences become particularly important when applications interact with an RTOS. Resources such as task handles, queues, and mutexes are managed by the kernel. Applications must therefore follow rules governing their creation, use, and destruction. Rust bindings can express some of these rules through types and lifetimes, while C generally relies on API contracts and programmer discipline.

However, Rust cannot automatically verify the behaviour of external C code. Operations such as dereferencing raw pointers and calling unsafe functions require explicit **unsafe** contexts. Within these contexts, the programmer must uphold safety conditions that the compiler cannot verify [14].

This is particularly relevant to the **foreign-function interface** (FFI), which allows Rust to interact with C code. Both languages must agree on calling conventions, data representation, and resource ownership. Rust's `extern "C"` declarations specify the calling convention but do not guarantee that the underlying C functions behave correctly [15].

Rust bindings may hide these unsafe operations behind a safe API. Such an interface allows application code to interact with the kernel without directly using `unsafe`. However, its safety depends on whether the binding implementation correctly maintains the required conditions.

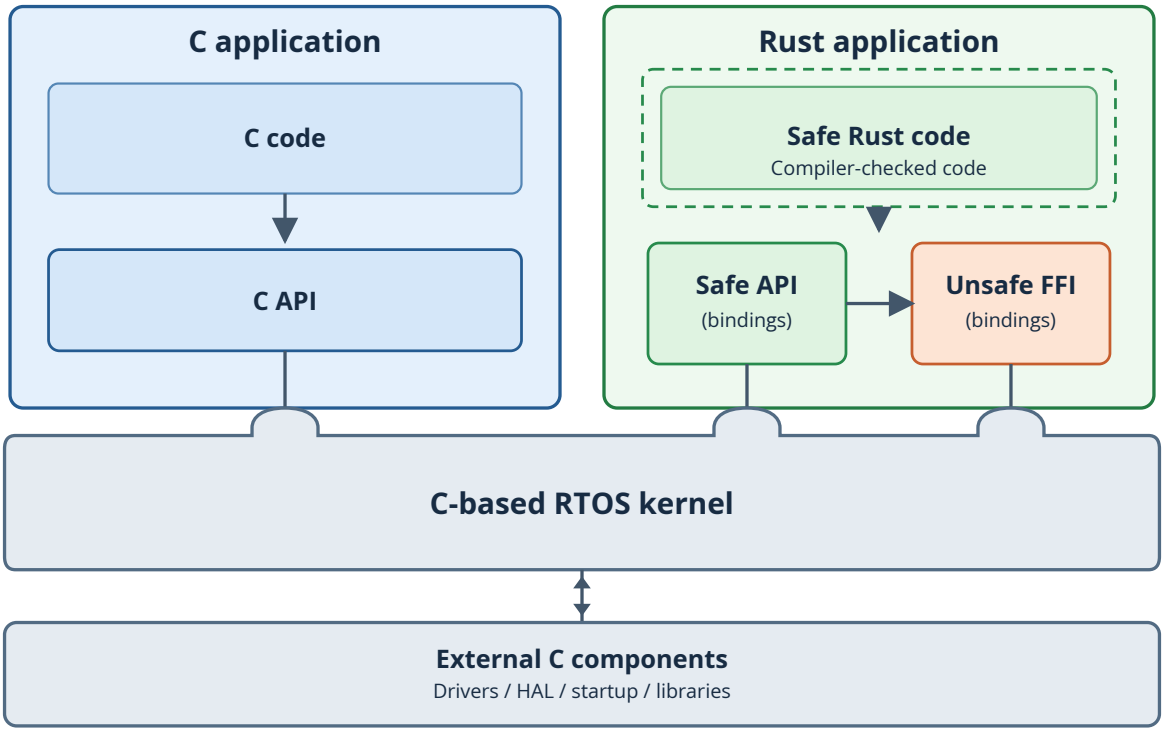


Fig. 5: C and Rust kernel interfaces

Fig. 5 illustrates how C and Rust applications can interact with a shared C-based RTOS. While C accesses the kernel through its API, Rust may use a safe binding interface built on unsafe FFI calls. Both approaches depend on kernel and external C code that Rust's compiler cannot verify.

Language bindings may introduce additional function calls, data conversions, or resource management that influence measured performance.

> **Key takeaway**
>
> Rust's compile-time safety guarantees do not automatically extend through FFI to a C-based RTOS. Both the safety boundaries and potential performance costs of language bindings must be considered when comparing C and Rust.

2.6 Software Complexity and Maintainability

Besides performance, memory usage and safety, the effort required to understand and maintain software is important in embedded systems. Source-code metrics provide indicators of this effort, although they cannot measure maintainability directly [16].

Source lines of code describe the amount of code that needs to be implemented and maintained. Cyclomatic complexity measures independent paths through a program's control flow and is commonly used to assess testing complexity [17]. Cognitive complexity considers how difficult the control flow is to understand [18]. Module coupling describes dependencies between software components, which can increase the effort needed to make changes [19].

Together, these metrics provide different perspectives on implementation complexity. They are particularly relevant when comparing C and Rust applications that perform the same tasks but require different adapters, interfaces and supporting code. Lower metric values do not necessarily mean that software is easier to maintain, but they help identify differences that warrant closer examination.

3 State of the Art

Previous research has examined embedded RTOS performance, compared programming languages on microcontrollers, and investigated Rust's memory-safety guarantees. However, these studies often focus on individual kernels, programming languages, or integration approaches.

This chapter reviews established RTOS benchmarks, empirical comparisons, and research on Rust interoperability. It examines their methods and limitations to identify the research gap addressed by this thesis.

3.1 RTOS Benchmarking Approaches

Comparing RTOS performance requires benchmarks that measure defined operations under reproducible conditions. One early approach is the **Rhealstone benchmark**, introduced by Kar and Porter in 1989. It evaluates six timing measurements intended to characterise the real-time behaviour of a processor and its operating system [20].

Rhealstone defines six components [21]:

- **Task switching:** The time required to switch between two tasks of equal priority.
- **Preemption:** The time required for a higher-priority task to take over from a lower-priority task.
- **Interrupt latency:** The delay between an interrupt request and the beginning of its service routine.
- **Semaphore shuffling:** The kernel overhead associated with transferring a semaphore between tasks.
- **Deadlock breaking:** The time associated with resolving priority-related resource contention.
- **Inter-task message latency:** The time required to transfer a message between tasks.

These components cover scheduling, interrupt handling, synchronisation, and communication. Rhealstone combines their timings into a single figure of merit, where lower operation times produce a higher score. However, the results depend on the precise measurement boundaries. The benchmark also does not directly describe application-level performance or memory consumption, and an aggregate score can hide differences between individual operations.

While Rhealstone focuses on the latency of individual operations, later approaches consider other aspects of RTOS performance. **Thread-Metric**, for example, evaluates

scheduling, communication, synchronisation, and memory allocation through operation rates [22]. This provides a complementary perspective by showing how many operations a system can complete within a given period.

Beyond the choice of performance metrics, comparing different RTOSs also requires a consistent benchmark implementation. **RTOSBench** addresses this challenge through an abstraction layer that connects common benchmark workloads to kernel-specific interfaces [23]. This allows tests to be reused across operating systems, although differences between adapters can still influence the results.

> **Key takeaway**
>
> Rhealstone provides reference operations, Thread-Metric adds throughput measurements, and RTOSBench addresses cross-kernel comparability. Together, they inform the benchmark design used in this thesis without requiring full conformance to any existing suite.

The specific workloads and measurement procedures used in this thesis are described in the methodology chapter.

3.2 Comparisons of Real-Time Operating Systems

Beyond defining how RTOS performance can be measured, previous research has applied similar metrics to compare operating systems and concurrency frameworks. Belleza and de Freitas compare C-based RTOS kernels using synchronisation, message passing, and event-handling operations [24]. These measurements cover several of the same areas as Rhealstone, but primarily reveal differences between kernel implementations rather than programming languages.

More recent studies also consider Rust-based systems. Vandervelden et al. compare several operating systems and concurrency frameworks [6]. These include Tock, Hubris, RTIC, and Embassy. Their evaluation considers scheduling and interrupt latency, communication, memory use, and isolation.

Wang extends this perspective by comparing the C-based FreeRTOS kernel with the Rust-based RTIC and Embassy frameworks on STM32F3 hardware. The study measures execution time and memory footprint [25].

However, these systems differ in their architecture and scheduling behaviour. For example, RTIC uses priority-based preemption, while Embassy primarily relies on cooperative asynchronous task scheduling. Such differences influence response times and scheduling overhead independently of the programming language. Consequently, these studies cannot isolate the performance impact of C versus Rust alone.

Beyond direct comparisons, individual Rust-based operating systems have also been studied in more detail. Tock, for example, combines Rust's type system with hardware protection to isolate applications and manage shared memory [26]. Building on this architecture, Culic et al. investigate how interrupt responsiveness can be improved while maintaining Tock's safety properties [27].

Ariel OS takes a different approach, focusing on an embedded Rust operating system evaluated through microbenchmarks across Arm Cortex-M, RISC-V, and Xtensa platforms [28]. These studies demonstrate different design choices and performance characteristics of Rust-based operating systems.

> **Key takeaway**
>
> Existing RTOS comparisons reveal differences in scheduling, communication, and memory usage. However, the systems also differ in architecture and implementation. Their results therefore cannot isolate the performance impact of C versus Rust.

3.3 C and Rust in Embedded Systems

While the previous studies focus on operating systems and their architectures, other research compares programming languages directly. Plauska, Liutkevičius, and Janavičiūtė compare C/C++, MicroPython, Rust, and TinyGo on an ESP32. They use computational workloads such as CRC32, SHA256, FFT, IIR, and FIR with shared inputs and expected outputs [29]. These workloads reveal differences in computation but do not measure RTOS scheduling or interrupt response.

Embuena et al. compare languages and platforms in terms of performance and power consumption, including STM32F4 Discovery and ESP32/RISC-V hardware [30]. Because optimisation settings are not matched, differences in build configuration also need to be considered when interpreting the results.

Thapa et al. study a more controlled C-versus-Rust development scenario. Two teams independently implement equivalent microcontroller functionality and compare performance, memory footprint, and development effort [31]. This is particularly relevant to an application-language comparison. However, the study does not evaluate equivalent workloads over the same real-time kernel. The study was also presented at IEEE DCOSS-IoT 2026.

Together, these studies demonstrate that C and Rust can be compared on embedded hardware. However, they do not establish how the two languages perform when the RTOS, board, and workload are held constant.

Performance and resource consumption are only part of the comparison between C and Rust. Another important difference concerns memory safety. Rust prevents many memory-related errors through ownership. However, its guarantees depend on the correctness of underlying abstractions. RustBelt formalises this relationship and shows why a safe interface must also have a sound implementation [32]. Research on unsafe Rust illustrates how difficult maintaining these requirements can be in practice [33].

Even when these requirements are satisfied, memory safety does not guarantee overall software correctness. Santoso, Kwee, and Chua examine which vulnerability classes Rust's memory model prevents and which remain possible [34]. Zhang et al. support this distinction through an analysis of 790 fixes in Rust programs. Many of the identified defects fall outside memory safety [35].

These limitations become especially relevant when Rust interacts with C code. At this boundary, Rust cannot verify whether external functions respect the assumptions made by its type system. McCormack, Sunshine, and Aldrich demonstrate the consequences by identifying undefined or undesirable behaviour in mixed-language interfaces [36]. Similar challenges arise when integrating Rust into larger C codebases, as illustrated by Rust for Linux [37]. In safety-critical systems, such dependencies also complicate the evidence required to establish system safety [38].

These challenges have motivated research into detecting errors that Rust's compiler cannot prevent. SafeFFI investigates sanitisation at language boundaries [39]. Miri detects certain forms of undefined behaviour, including invalid pointer operations and data races [40]. Other approaches focus on identifying memory-safety defects in unsafe code, including use-after-free and double-free errors [41], [42]. Although these tools improve error detection, they cannot guarantee that every external interface behaves correctly.

> **Key takeaway**
>
> Existing language comparisons provide evidence about performance and resource consumption, but rarely isolate RTOS behaviour. Rust improves memory safety without guaranteeing overall correctness, particularly when external C code is involved.

These findings are particularly relevant when Rust applications use C-based RTOSs. In such systems, language bindings connect Rust code to the kernel. Their implementation affects both memory safety and performance.

3.4 Rust Applications on C-Based Kernels

Several projects provide ways to use Rust with existing C-based RTOSs. However, support and integration approaches differ between systems. Sharma et al. identify common

interoperability challenges through an analysis of Rust packages, tools, and developer experiences [8].

The C-based RTOSs considered in this thesis provide different integration routes. FreeRTOS-rust offers wrappers and a shim for the FreeRTOS API [43]. Zephyr documents Rust as an optional language module requiring suitable build configuration and target support [44]. NuttX documents experimental Rust support, including C interoperability and selected processor architectures [45]. These resources describe how Rust can be integrated, but do not themselves provide controlled C-versus-Rust performance comparisons.

Another approach to integrating Rust with a C-based RTOS is TECS/Rust. It connects Rust components to an existing RTOS through a component framework. Yoshimura, Oyama, and Azumi evaluate this approach and report low execution-time overhead [46]. Later work improves synchronisation through memory-safe locking mechanisms [47]. The approach has also been extended to multiprocessor systems [48].

These studies show that Rust can be integrated into a C-based RTOS with limited overhead in the systems evaluated. However, they focus on one integration approach rather than comparing equivalent C and Rust applications across different kernels.

Performance is also not the only consideration. Motogna, Vescan, and Şerban identify safety, security, and maintainability as important qualities of embedded software [49]. This makes the integration effort, language bindings, and safety requirements relevant alongside timing and memory consumption.

The studies reviewed in this chapter therefore leave an important question unanswered. RTOS comparisons mainly examine differences between kernels or frameworks. Language comparisons often use computational workloads without an RTOS. Meanwhile, Rust integration studies focus on specific implementations. As a result, there is limited evidence on how C and Rust compare when the underlying RTOS, hardware, and workload remain unchanged.

The limitations of previous research are summarised in Table I. The first column groups studies by their approach, while the second identifies what each approach leaves unresolved. The final column shows how this thesis addresses these limitations.

Approach	What remains unresolved	This thesis
C RTOS comparisons (Belleza)	Kernel effects are compared, not application-language effects	Compares C and Rust on a fixed kernel
Rust OS comparisons (Vandervelden, Wang)	Kernel or framework also varies	Matches kernel and workload
Individual Rust systems (Levy, Frank, Culic)	No matched C application comparison	Uses paired application implementations
Languages on hardware (Plauska, Embuena)	RTOS operations are not isolated under matched conditions	Benchmarks RTOS operations
Industrial C/Rust comparison (Thapa)	Does not isolate operation-level RTOS costs	Measures defined RTOS operations
Rust/C integration (Yoshimura)	Evidence is specific to one integration approach	Examines several kernels and targets

Table I: Comparison approaches and research gap

The main contribution of this thesis is a controlled comparison of C and Rust applications on the same hardware and RTOS. Workloads and relevant build settings are kept consistent within each experiment. This allows timing, memory consumption, and integration costs to be compared under similar conditions.

The next chapter describes the experimental setup; the Methodology chapter then explains the measurement and analysis procedures.

4 Experimental Setup

This chapter describes the hardware and software environment used for the experiments. It introduces the four development boards and their timing facilities, defines the evaluated RTOS configurations, and describes the common benchmarking framework. It also covers the build environment and the serial communication protocol; the measurement and comparison procedures are described in the following chapter.

4.1 Hardware Platforms

The evaluation uses four development boards with Arm Cortex-M and RISC-V processors. Their main specifications are summarised in Table II. All timing measurements are performed on the devices themselves. The host is responsible for building, flashing and collecting the results.

Board	Microcontroller	Core	Max clock	Flash	SRAM
STM32F4 Discovery	STM32F407VGT6	Cortex-M4F	168 MHz	1 MiB	192 KiB
FK723M1 core board	STM32H723ZGT6	Cortex-M7	550 MHz	1 MiB	564 KiB
ESP32-C6 DevKitC	ESP32-C6	RISC-V	160 MHz	8 MiB (ext.)	512 KiB
nRF52840 DK	nRF52840	Cortex-M4F	64 MHz	1 MiB	256 KiB

Table II: Hardware platforms used in the evaluation. Clock frequencies are rated maxima.

The **STM32F4 Discovery** uses its onboard ST-LINK/V2 interface for programming and debugging. Benchmark communication takes place through USART2 on PA2 and PA3 using an external CP2102 USB-to-UART bridge. FreeRTOS and NuttX use the Data Watchpoint and Trace (DWT) cycle counter for timing measurements, while Zephyr uses the Cortex-M SysTick counter. Hardware timers support interrupt-related experiments [50].

The **FK723M1 core board** operates at 520 MHz in the evaluated configuration, below its rated maximum of 550 MHz [51]. Programming and debugging require an external ST-LINK/V2 probe. The benchmark console uses USART1 on PA9 and PA10 through a CP2102 bridge at 115200 baud. FreeRTOS and NuttX use the DWT cycle counter for timing measurements, while Zephyr uses the Cortex-M SysTick counter. The Zephyr configuration uses the upstream `fk723m1_zgt6` board target with a 25 MHz crystal and a dedicated clock overlay.

The **ESP32-C6 DevKitC** uses its integrated USB Serial/JTAG peripheral for programming, debugging and benchmark communication [52]. Timing measurements use the counter facilities provided by the corresponding RTOS port.

The **nRF52840 DK** provides SWD debugging and a virtual serial port through its onboard J-Link interface [53]. The benchmark console uses UART0 on P0.06 and P0.08. Timing measurements use the DWT cycle counter, which must be re-enabled when necessary after low-power transitions.

4.2 Timing Sources

The measurement-counter frequency is not always the same as the processor clock frequency. On the Arm boards, FreeRTOS and NuttX use the DWT cycle counter, while Zephyr uses the Cortex-M SysTick counter on the STM32 boards and DWT on the nRF52840. The ESP32-C6 uses different timing sources depending on the RTOS. Table III summarises the counter frequencies used to convert recorded ticks into time units.

Platform	RTOS	Timing source	Frequency
STM32F4	FreeRTOS, NuttX	DWT CYCCNT	168 MHz
STM32F4	Zephyr	SysTick-based 64-bit software counter	168 MHz
STM32H723	FreeRTOS, NuttX	DWT CYCCNT	520 MHz
STM32H723	Zephyr	SysTick-based 64-bit software counter	520 MHz
nRF52840	FreeRTOS, Zephyr, NuttX	DWT CYCCNT	64 MHz
ESP32-C6	FreeRTOS	SYSTIMER	16 MHz
ESP32-C6	Zephyr	SYSTIMER	16 MHz
ESP32-C6	NuttX	RISC-V performance counter	160 MHz

Table III: Timing sources and counter frequencies used in the matched configurations.

The ports provide the timing sources and handle hardware-specific requirements. Counter rollover occurs when a finite-width counter reaches its limit and wraps; elapsed-time calculations must account for this wrap. Counters may also need to be re-enabled after low-power transitions.

4.3 RTOS Configurations

The evaluation covers five RTOS kernels across 27 configurations. FreeRTOS, Zephyr and NuttX are each evaluated with C and Rust applications on all four hardware platforms, forming 12 matched C/Rust pairs. The two Rust-native kernels are evaluated separately: FreeRTOS-in-Rust on the STM32F4 and STM32H723, and Rivet on the STM32F4. These three additional configurations are used for kernel-level comparisons rather than direct application-language comparisons.

Ten additional systems were integrated during development but excluded from the final evaluation: Tock, Hubris, Embassy, RTIC, Ariel OS, RIOT OS, RT-Thread, ChibiOS/RT, Eclipse ThreadX and RTEMS. Their differing execution models, unsupported benchmark operations or incomplete C/Rust implementations prevented sufficiently comparable measurements.

4.4 Benchmarking Framework

To compare C and Rust across different hardware platforms and RTOS kernels, a common benchmarking framework was developed. It separates experiment management on the host from workload execution on the target device. This allows the same benchmark definitions to be used across configurations while excluding host-side communication from timing measurements.

The framework consists of a command-line interface, a core library, system-specific adapters, and benchmark runners written in C and Rust. The host manages experiment configurations, builds and flashes firmware, starts measurement runs, and validates the results. The adapters handle platform-specific build and flashing procedures, while the device-side runners execute the workloads and record timing values using hardware counters.

On the device, the workloads are implemented in shared C and Rust runners. Both use a port abstraction to access kernel operations and timing facilities. The C implementation uses a table of callbacks, while Rust uses a trait. This separates the benchmark logic from the underlying RTOS and board-specific code.

Timing is performed either directly in the runner or within a port-specific measurement routine, depending on the operation.

4.5 Build Environment

To support reproducibility and traceability, each build uses an isolated directory and is identified by its source revisions, target architecture, programming language, configuration, benchmark version, and compiler. Cached firmware is reused only when these properties match. Raw device output, validated measurements, and experiment metadata are retained so that results can be traced back to the configurations under which they were obtained.

The matched C and Rust configurations use nominal optimisation level 2: `-O2` for C and `opt-level=2` for Rust. These settings do not imply identical optimisation passes or generated code. The C benchmark code is compiled with `-O2`, while Rust uses release-profile optimisation level 2. The Rust code is compiled as a static library and linked into the firmware using the corresponding C toolchain. The underlying kernels retain their own build configurations, so the application and kernel code do not necessarily use identical optimisation settings. The relevant compiler versions and build settings are summarised in Table IV.

Component	Compiler / setting	Value
C, Arm boards	GCC	14.2.1
C, ESP32-C6	GCC	14.2.0
Rust, all boards	rustc	1.98.1
C benchmark applications	Optimisation	`-02`
Rust applications	Optimisation	`opt-level=2`
Rust applications	Panic strategy	`panic=abort`
Rust applications	Code generation units	`codegen-units=1`
Rust applications	Rust LTO	Enabled
NuttX and Zephyr kernels	Optimisation	`-0s`

Table IV: Compiler versions and relevant optimisation settings for the matched C/Rust configurations. Rust is linked as a static library; its LTO setting does not imply cross-language LTO.

4.6 Communication Protocol

The host and target device exchange commands and measurement data through a line-oriented serial protocol using JSON messages. Each message includes a protocol version and type, allowing benchmark data to be distinguished from unrelated console output. Communication follows a defined sequence, beginning with a handshake to establish the device identity and available benchmarks. The host sends a `hello` message containing a random challenge, which the device answers with a `ready` response. Once the handshake is complete, a `start` request specifies the workloads and sample counts. The device then reports individual measurements and signals the end of the run with a `complete` message.

The device validates incoming requests, while the host verifies device identity, sample order, sample counts, and successful completion.

Samples are buffered and transmitted after the timed operations.

5 Methodology

The experiments use the hardware, RTOS configurations and benchmarking framework described in the preceding chapter. The methodology is designed to distinguish differences between C and Rust from those caused by the hardware or RTOS kernel. For the primary comparison, both languages implement the same benchmark workloads on the same hardware and kernel. This allows the application implementations and their kernel interfaces to be compared under otherwise similar conditions.

The evaluation focuses on individual RTOS operations rather than complete applications. Each benchmark uses defined workloads and measurement boundaries to support consistent comparisons. Rust-native kernels are evaluated separately, as they do not provide equivalent C and Rust application configurations.

5.1 Benchmark Suite

The benchmark suite contains 26 cases covering the main operations and timing characteristics of an RTOS. Several are based on operations considered by Rhealstone, while others examine communication, interrupt behaviour, CPU usage and memory allocation [21]. The suite is not intended to conform to an existing benchmark standard.

Table V lists the cases and their measured quantities. The following descriptions explain the timing boundaries and workload parameters where these are important for interpreting the results.

No.	Benchmark	Measured quantity	Reported unit
1	timer_overhead	Timer and runner overhead	µs
2	queue_roundtrip	Queue send and receive	µs
3	semaphore_latency	Semaphore give and take	µs
4	mutex_uncontended	Mutex lock and unlock	µs
5	alloc_latency	Allocation and release of 64 bytes	µs
6	task_handoff	Task release to receiver execution	µs
7	yield_roundtrip	Voluntary context-switch round trip	µs
8	preempt_latency	Higher-priority task wake-up	µs
9	mutex_priority_inheritance	Waiting time under priority inversion	µs
10	scheduler_overhead	Scheduler call without a task switch	µs
11	notification_latency	Task notification to resumption	µs
12	ipc_pingpong	Request-and-reply latency	µs
13	ipc_payload_sweep	Latency across message sizes	µs
14	ipc_throughput	Sustained message transfer rate	B/s
15	zero_copy_ipc	Saving from reference-based transfer	ticks
16	isr_to_task	Interrupt handler to task resumption	µs
17	trigger_to_isr	Interrupt request to handler entry	µs
18	nested_irq_overhead	Additional time from interrupt nesting	µs
19	tick_overhead	Execution time of one kernel tick	µs
20	max_interrupt_disabled	Interrupt delay caused by masking	µs
21	periodic_jitter	Variation in periodic wake intervals	µs
22	periodic_jitter_loaded	Periodic release error under load	µs
23	sleep_accuracy	Error of a requested sleep	µs
24	idle_cpu_overhead	Tick processing share while idle	ppm
25	cpu_overhead_under_load	Non-idle CPU share under load	ppm
26	heap_fragmentation	Fragmentation of available heap memory	ppm

Table V: Overview of the 26 benchmark cases, their measured quantities and reported units.

Timing measurements are recorded as raw counter ticks and converted to microseconds using each configuration's recorded counter frequency. Throughput is derived from the elapsed ticks of the fixed 1 MiB transfer.

The benchmarks cover different aspects of RTOS performance. Some measure the cost of individual kernel operations, while others involve task scheduling, communication or interrupt handling. The suite also examines periodic timing, CPU usage and memory allocation.

Most timing measurements include more than the kernel operation itself. They may also include API calls, language bindings, scheduling decisions or timer access. The results therefore describe the complete measured operation rather than the isolated cost of a kernel primitive.

The workloads are designed to be comparable, but their implementation may differ between kernels due to differences in synchronisation mechanisms, interrupt handling and load generation. These differences can influence measured timings independently of the programming language. Therefore, direct C and Rust comparisons are restricted to

the same kernel, while comparisons between kernels must also account for differences in their implementations. The exact workloads, timing boundaries, parameters and checks for all 26 cases are documented in Appendix A.

5.2 Measurement Procedure

The host and target device validate the command and measurement exchange described in Section 4.6. Invalid messages, incomplete runs, and timeouts are treated as failures rather than valid measurements. All received data is retained for later inspection.

The queue round trip, for example, measures a send and receive between two timestamp readings, while interrupt measurements require timestamps inside handlers or resumed tasks. Some hardware-dependent measurements use shared C routines for both language paths. In these cases, parts of the measurement or control procedure are implemented in C and used by both language configurations. The resulting C/Rust comparison therefore includes a shared implementation rather than two fully independent versions of the measured operation. This is distinct from using the same timer interface while implementing the workload separately in each language. The affected benchmark categories are described in Appendix A. Timing measurements are recorded as raw timer ticks and converted to microseconds using the recorded counter frequency. Throughput is reported in bytes per second (B/s) after converting each valid observation, while CPU utilisation is reported in parts per million (ppm).

The measured intervals are reported without subtracting timer overhead, as they represent the complete execution paths, including the instrumentation required to obtain them. Buffering prevents serial communication from affecting the measurements.

Each supported benchmark is configured for ten warm-up iterations followed by 100 recorded observations per run. Ten runs provide 1,000 observations per case before division into parameter variants and exclusion of invalid or sentinel observations.

For benchmarks with several parameter values, the samples are divided between them. For example, a benchmark with five payload sizes records 20 samples per size in each run. Across ten runs, this gives 200 samples per size.

5.3 Statistical Evaluation

The analysis computes the median, the 95th, 99th and 99.9th percentiles (p95, p99 and p99.9), and the highest observed value. Appendix B presents the median and p99; the other summaries remain in the analysis output. Percentiles use linear interpolation between ordered observations. Samples from all ten runs are combined for each case and parameter value. Percentiles describe the upper tail of the measured quantity. For latency and other time measurements, higher percentiles represent longer observed durations,

and the difference between p99 and the median describes the spread of slower observations. For throughput, p99 represents higher observed transfer rates and therefore the high-throughput tail; it does not describe unfavourable throughput. Throughput values are reported in B/s after converting each valid observation from its elapsed ticks. These statistics describe the observed results. They do not establish worst-case execution times. This is particularly important for p99.9, which is close to the highest observed value when only 200 samples are available. For the loaded periodic-jitter benchmark, missed releases are counted separately and excluded from the latency statistics.

5.4 Resource Measurement

Memory usage is evaluated using both the firmware image and measurements from the running system. Static memory usage is determined for each build, while runtime measurements are collected after the benchmarks finish. Runtime memory values are reported in bytes; static footprint values are reported in KiB.

Runtime memory usage is reported by the device in its completion message, outside the timed measurements. The framework records the benchmark task's stack high-water mark (the minimum unused stack space observed during execution), the free heap at the end of the run and, where available, the minimum free heap observed during execution. It also measures allocation overhead by comparing the requested size of a one-byte allocation with the memory consumed by the allocator. If a kernel does not expose the information required for a measurement through its API, the corresponding value is omitted. Failed task-creation and allocation-overhead measurements may instead be reported as zero, making them indistinguishable from valid zero values. No such ambiguous values occurred in the original measurement campaign.

The framework also estimates the memory overhead of creating a task. Where task creation is heap-accounted, it creates a temporary task and measures the resulting heap change. The requested stack size is subtracted, so the reported value includes the task control block and allocator metadata; on NuttX, it also includes additional task-creation bookkeeping. The temporary task is then deleted. Where a minimum free heap value is reported, it is sampled before this test. For kernels with statically allocated task objects, such as Zephyr and Rivet, the reported value is the size of the task structure alone. These values are therefore comparable between C and Rust on the same kernel, but not directly across different kernels.

To determine the static footprint, the host examines the firmware image directly rather than relying on a toolchain size report. It distinguishes code, read-only data, initialised data and zero-initialised data. Read-only data contributes to flash usage, while initialised data contributes to both flash and RAM. Zero-initialised data occupies RAM but does not require storage for its contents in flash.

Finally, the framework records the number of active tasks and the minimum and maximum free stack space among them. These are stored as aggregate values rather than separate records for every task.

5.5 Safety Analysis

Besides performance and memory consumption, this thesis examines how memory-safety responsibilities differ between C and Rust implementations. The analysis identifies operations that may affect memory safety if used incorrectly. These include pointer dereferences, field and array accesses, pointer arithmetic and conversions, allocation and deallocation, accesses to mutable global state and calls to RTOS APIs. For Rust, the analysis also records whether operations occur within `unsafe` contexts.

To investigate these operations and how individual benchmarks access the RTOS, a dedicated static-analysis pipeline was developed. Existing tools such as Clippy, Clang Static Analyzer, Miri and CodeQL do not directly provide the combination of source-level operation counts and benchmark-specific tracing required for this study.

The analysis begins by parsing the project-written source code into abstract syntax trees (ASTs). These trees represent the structure of the code, allowing operations such as pointer dereferences and function calls to be identified without relying on text searches. C files are parsed using libclang with the compiler definitions and include paths of the corresponding build. Rust files are parsed using the syn library and examined with a custom extractor.

For Rust, an operation is classified as occurring within an `unsafe` context when it appears inside an `unsafe` block or function. The reported percentage is calculated by dividing the number of active, resolved operation records within these contexts by the total number of active, resolved operation records for the relevant source-code role. This measures the location of recorded operations, not the proportion of source code that is unsafe.

For each identified operation, the analysis records its type and location in the source code. For example, the analysis identifies `value = *ptr;` as a pointer dereference and records its file and line number. The location determines which part of the implementation the operation belongs to; if it occurs in the FreeRTOS integration code, it is attributed to that source-code role. This allows safety-relevant operations to be counted and compared separately across different parts of the implementations.

The next step examines how individual benchmarks access the RTOS. For each RTOS and board combination, a profile is generated from the benchmark definitions, source code and build information. A Python script examines the benchmark mappings and function definitions to identify the entry points of individual tests. It combines these with compiler

settings and RTOS-specific rules to determine the relevant source files, bridge functions and RTOS interfaces.

The analysis uses these profiles to trace function calls from each benchmark towards the RTOS interface. C and Rust call paths are examined separately. Calls from Rust into C are connected by matching FFI declarations to their corresponding C functions. For example, tracing the FreeRTOS queue benchmark identifies the RTOS functions used to create and delete queues, as well as send and receive messages.

Not all call paths can be identified through direct function calls. Callbacks are included when their registrations can be resolved, while interrupt handlers are traced separately when they can be identified from the source code or platform configuration. Calls that cannot be resolved are marked accordingly in the tracing results. Since the tracing examines source code without executing it, the identified paths do not necessarily represent which functions are called at runtime.

The tracing results record function calls, their arguments and associated checks for each benchmark. They distinguish calls from the shared benchmark runner from those in the RTOS integration code and identify calls targeting RTOS services. These results complement the earlier source-level analysis, where safety-relevant operations are counted by firmware configuration and source-code component rather than by individual benchmark.

The analysis is limited to project-written benchmark and integration code. The internal implementations of third-party RTOS kernels and hardware-abstraction libraries are excluded. Operations that cannot be resolved are recorded separately and excluded from the confirmed operation counts. These counts show where safety-relevant operations occur in the source code, but not whether they are used incorrectly. Therefore, the analysis cannot establish the memory safety of a complete firmware system.

5.6 Complexity and Maintainability

To assess the effort involved in implementing and maintaining the C and Rust configurations, the study uses the four source-code metrics introduced in Section 2.6. The analysis covers project-written benchmark code, including shared runners and RTOS adapters. Third-party kernel implementations are excluded.

The metrics are calculated for individual modules. For Rust, the application and RTOS boundary modules are analysed separately. For C, the benchmark logic and supporting functions are analysed together because they share a source file. Consequently, the measured modules do not always represent identical boundaries in both languages. The results are therefore interpreted in relation to the code included in each module.

Code size is measured in source lines of code. Cyclomatic complexity measures the number of independent control-flow paths through a function. It is calculated as one plus

the number of decision points. Cognitive complexity measures how difficult a function is to follow by assigning points to control-flow structures such as conditionals and loops. Additional points are assigned for nesting, so deeply nested structures receive higher scores. Both complexity metrics are reported using the median, mean and maximum across the functions in each module.

Module coupling is estimated by counting distinct dependencies declared through C `#include` directives and Rust `use` and `mod` statements. Dependencies are separated into internal and external ones. This provides a simple measure of declared dependencies, but does not represent the complete function-call or FFI graph.

Mozilla's rust-code-analysis provides the per-function cyclomatic and cognitive complexity values for both languages [54]. A Python script calculates source lines of code and the dependency-based coupling metric. The analysis operates on individual source files, so functions defined in C headers are attributed to those headers rather than to files that include them.

6 Results

The results compare C and Rust applications running on the same hardware and RTOS, while Rust-native RTOSs are considered separately. Not every RTOS configuration supports all 26 benchmark cases. The main gaps concern tick accounting, CPU-load measurements and memory allocation. Within each matched board and RTOS, C and Rust support the same cases. The complete coverage by configuration is provided in Appendix B.

6.1 Benchmark Measurements

The benchmark measurements are presented in five RTOS-specific tables in Appendix B: Table VIII, Table IX, Table X, Table XI and Table XII. Fig. 6 provides an overview of the median Rust/C ratios for the matched configurations. A ratio of 1 indicates identical median values, while values above or below 1 indicate higher or lower measurements for Rust. Since the benchmarks measure different quantities, these differences do not necessarily indicate better or worse performance.

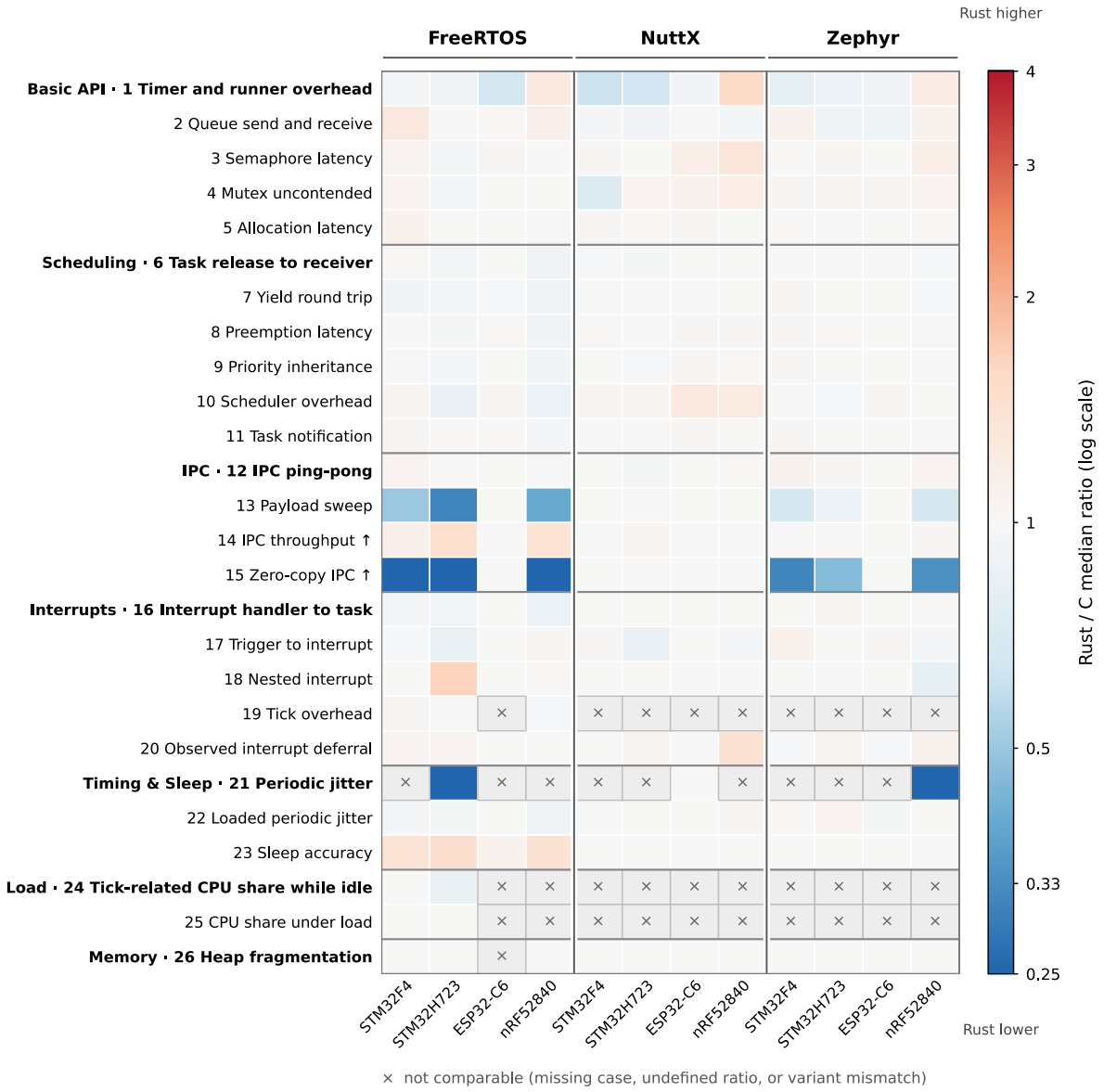


Fig. 6: Median Rust/C benchmark ratios across matched RTOS and hardware configurations.

Scheduling, synchronisation and interrupt benchmarks generally show similar median values for C and Rust on the same hardware and RTOS. However, the medians do not always reflect the higher latencies observed in individual measurements. For example, on Zephyr ESP32-C6, preemption latency (8) has a median of 8.75 µs for both languages, while p99 reaches approximately 470 µs in both implementations. The increased tail latency is therefore observed for both languages rather than being specific to either implementation.

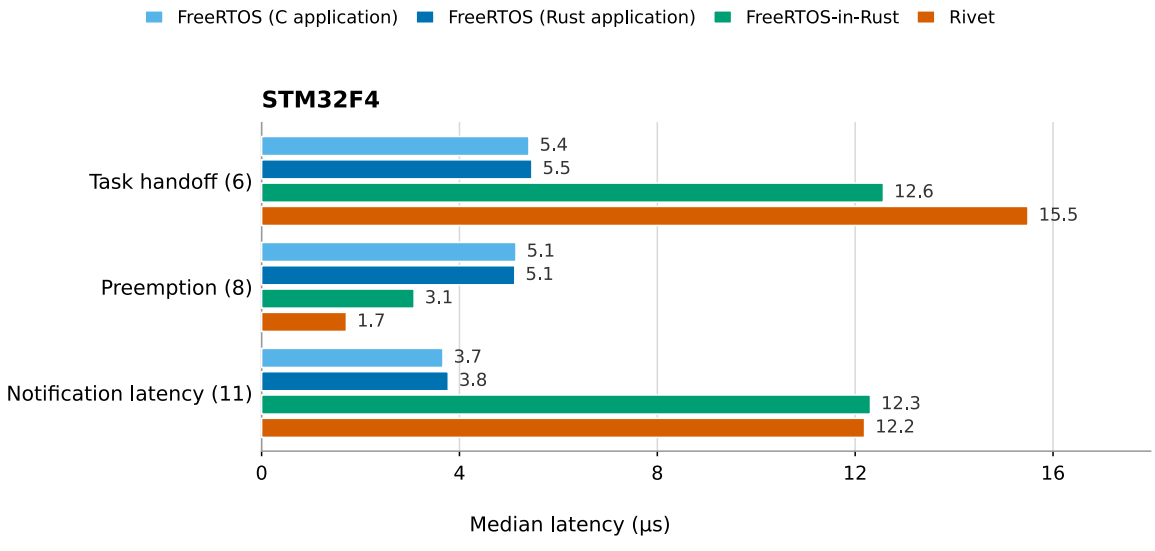
Inter-task communication shows some of the clearest differences between C and Rust, although the results depend on the operation being measured. On FreeRTOS STM32F4,

Rust achieves lower median payload-transfer latency (13) than C (26.8 versus 53.5 µs) and higher median IPC throughput (14) (3.92 versus 3.47 MB/s). This comparison uses the corrected per-sample conversion of protocol case 19, which times a fixed 1 MiB transfer. The difference includes RTOS, binding, adapter and shared-code effects and cannot be attributed to the programming language alone. The results vary across configurations, with zero-copy savings (15) showing no consistent advantage for either language.

For the payload-sweep benchmark, these values refer to the 256-byte payload, rather than an aggregate across all payload sizes. The median is calculated from 200 samples collected across ten runs for each language.

Loaded periodic jitter (22) requires particular care when comparing RTOSs because its load-generation mechanisms differ across platforms and boards. The reported percentiles describe only retained releases; some variants retain very few valid observations, as shown by their miss counts. Low survivor latencies must not be read as evidence of reliable release timing.

Fig. 7 compares selected scheduling latencies on STM32F4 between conventional FreeRTOS and the two Rust-native kernels.



Rust-native bars: latest retained run; FreeRTOS C/Rust: newest campaign.

Fig. 7: Selected scheduling latencies across FreeRTOS and Rust-native RTOS configurations on STM32F4. Release mechanisms differ; these values do not isolate kernel-language effects.

Both Rust-native kernels record lower median values for the preemption benchmark (8) than conventional FreeRTOS, but higher task-handoff (6) and notification latencies (11). These are configuration-level observations: FreeRTOS-in-Rust uses an interrupt-driven release, whereas conventional FreeRTOS releases a semaphore waiter. The comparison

therefore does not isolate a kernel-language effect. Rivet achieves the lowest preemption latency at 1.71 µs, compared with 3.08 µs for FreeRTOS-in-Rust and approximately 5.1 µs for conventional FreeRTOS. In contrast, the C and Rust applications on conventional FreeRTOS produce similar results across all three measurements.

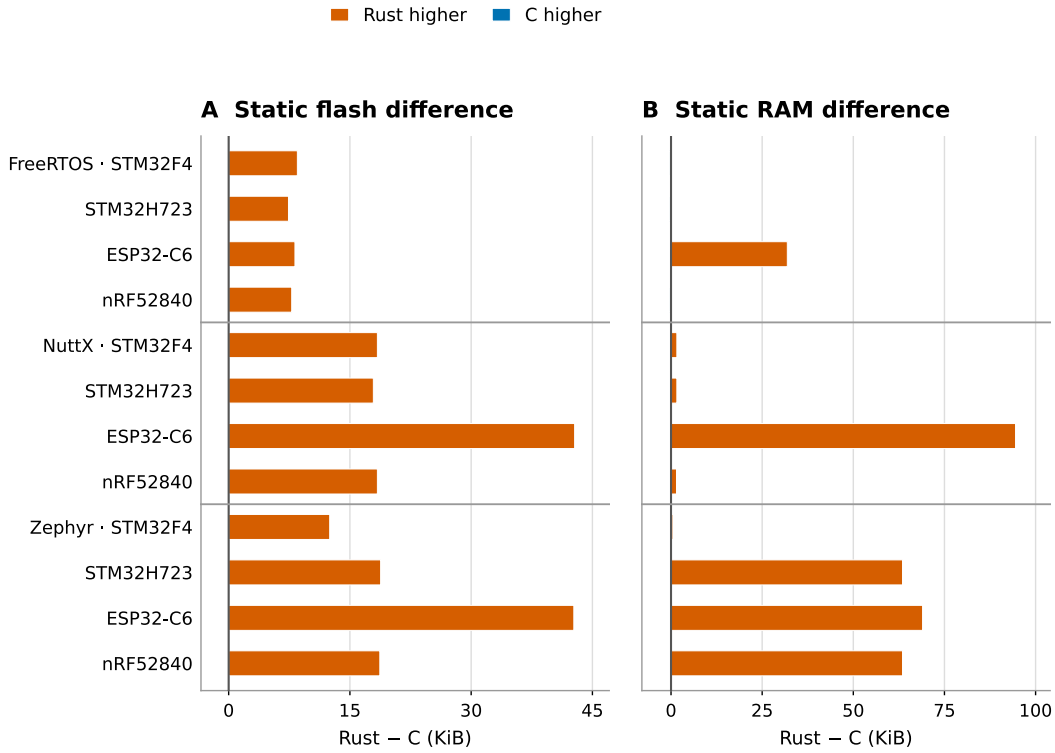
The two Rust-native kernels also differ in communication and interrupt handling. Rivet records lower queue round-trip (2) and IPC ping-pong (12) latencies. On STM32F4, preserved summary-derived medians indicate higher IPC throughput (14) for Rivet than FreeRTOS-in-Rust (6.53 versus 5.05 MiB/s); however, the raw supplementary native bundle is unavailable, so these medians are not independently reproduced sample-level statistics and p99 throughput is not reported. FreeRTOS-in-Rust on STM32H723 has a higher derived median (6.96 MiB/s), but that is a different board and is not a same-board comparison. FreeRTOS-in-Rust records lower trigger-to-interrupt latency (17).

6.2 Memory Usage

Memory is examined at two levels: runtime memory reported by the running benchmark, including heap high-water, stack headroom and per-task overhead, and the static memory contained in the linked ELF image. The complete measurements for all 27 firmware configurations are listed in Appendix C: Table XIII reports the runtime fields and Table XIV the section-based static footprint. Runtime and static memory are kept separate here, and the static values are image sizes rather than runtime heap or stack observations.

Within matched C and Rust pairs, per-task RAM overhead is identical on the same kernel and board, and heap allocation overhead and heap fragmentation are close. The benchmark task's stack high-water mark (the minimum unused stack observed) is greater for Rust on every matched pair. This measures remaining benchmark-task headroom, not total memory use, and does not offset the larger static footprint observed on several boards. These runtime differences are small compared with the static footprint.

Fig. 8 shows the static difference between Rust and C, calculated as the footprint of the C program subtracted from the footprint of the Rust version for each of the 12 matched configurations. Positive values mean that the Rust implementation uses more memory, negative ones mean that the C version uses more. Flash and RAM usage are measured and depicted separately.



ELF section differences (Rust − C); positive = Rust uses more, negative = C uses more.
RAM is linked static RAM, not runtime heap or stack.

Fig. 8: Static flash and RAM differences between matched Rust and C firmware configurations.

Static RAM differences vary by platform. On FreeRTOS, the C and Rust implementations use nearly identical amounts on the Arm boards, while Rust requires an additional 31.9 KiB on ESP32-C6. NuttX also shows small differences on the Arm boards, but Rust uses 94.5 KiB more RAM on ESP32-C6, an increase of approximately 30%. For Zephyr, memory usage is almost identical on STM32F4, whereas Rust requires an additional 63.5–69.0 KiB on the remaining boards.

The Rust-native kernels show a different memory footprint on STM32F4. FreeRTOS-in-Rust uses 38.0 KiB of flash and 78.3 KiB of static RAM, less than either conventional FreeRTOS configuration. Rivet uses more flash (78.3 KiB) but less static RAM (46.7 KiB) than both. These differences reflect complete firmware configurations rather than the kernel implementation language alone. The full measurements are provided in Appendix C.

6.3 Static Safety Analysis

The safety analysis examines how C and Rust implementations handle operations that may affect memory safety. Besides identifying these operations, it distinguishes between application code and the code used to interact with the RTOS. This allows the results to

show not only which operations occur, but also where they are located. The complete measurements are provided in Appendix D.

A clear difference emerges in how pointer operations are distributed. In the C implementations, pointer dereferences occur in both the benchmark applications and their RTOS adapters. The adapters contain more dereferences across all matched configurations, although the counts vary between boards. For example, the FreeRTOS adapter contains 47 occurrences on both STM32 boards but 142 on nRF52840. Pointer arithmetic follows a different pattern. It occurs mainly in the C applications, while the Zephyr and NuttX adapters contain none. The distribution therefore depends on the type of operation being examined.

The Rust implementations show a different result. Across all 12 matched configurations, their application code contains no resolved raw-pointer dereferences, pointer arithmetic or pointer conversions. Direct raw-pointer dereferences remain present in the FFI boundaries, however. Fig. 9 illustrates this difference in source-code organisation.

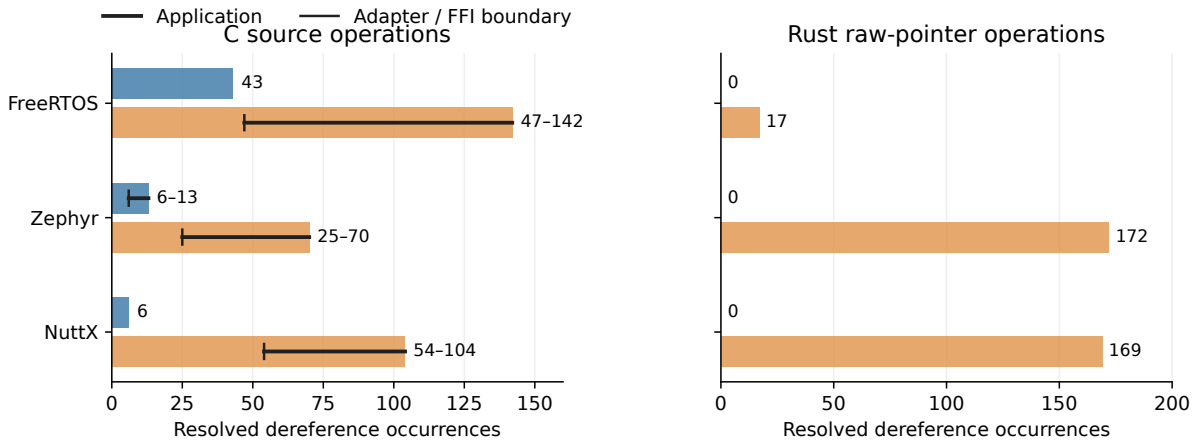


Fig. 9: Resolved pointer dereferences by application and integration role. Ranges span boards; C and Rust operation categories are not directly comparable.

The Rust FFI boundaries contain 17 resolved raw-pointer dereferences on FreeRTOS, 172 on Zephyr and 169 on NuttX. The latter two values are lower bounds because some boundary dereferences could not be resolved. These findings indicate that the analysed Rust applications avoid direct raw-pointer handling, while such operations remain necessary in their integration code. The Rust firmware also includes C support code, which is analysed separately in Appendix D. Because C and Rust use different syntax categories, the operation counts cannot be interpreted as a direct comparison of memory safety.

Furthermore, the absence of raw-pointer dereferences does not mean that the Rust applications avoid `unsafe` contexts. The analysis also records whether operations occur within such contexts, as shown in Fig. 10.

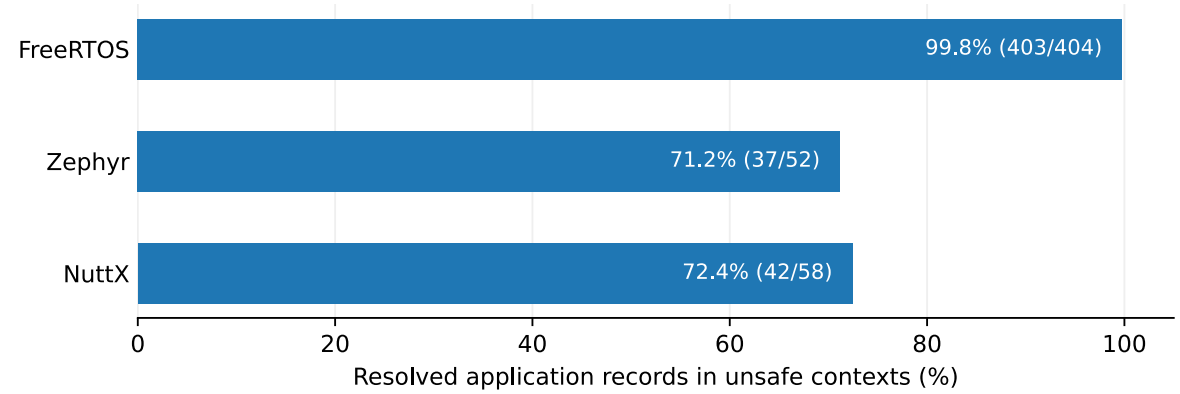


Fig. 10: Share of resolved Rust application operations inside unsafe contexts. This measures the location of recorded operations, not the proportion of unsafe source code.

The difference is particularly noticeable on FreeRTOS, where 99.8% of the resolved application operations occur within `unsafe` contexts, compared with approximately 71–72% on Zephyr and NuttX. This does not contradict the pointer results. The inventory includes RTOS calls and accesses to global state, so operations can be recorded inside `unsafe` contexts without involving raw-pointer dereferences. All resolved operation records in the Rust FFI boundaries also occur within `unsafe` contexts. The percentages therefore describe how the code is organised rather than how safe it is.

The Rust-native systems offer another perspective on this distinction. Fig. 11 compares conventional FreeRTOS with FreeRTOS-in-Rust, whose analysed project code also contains a high proportion of operations within `unsafe` contexts.

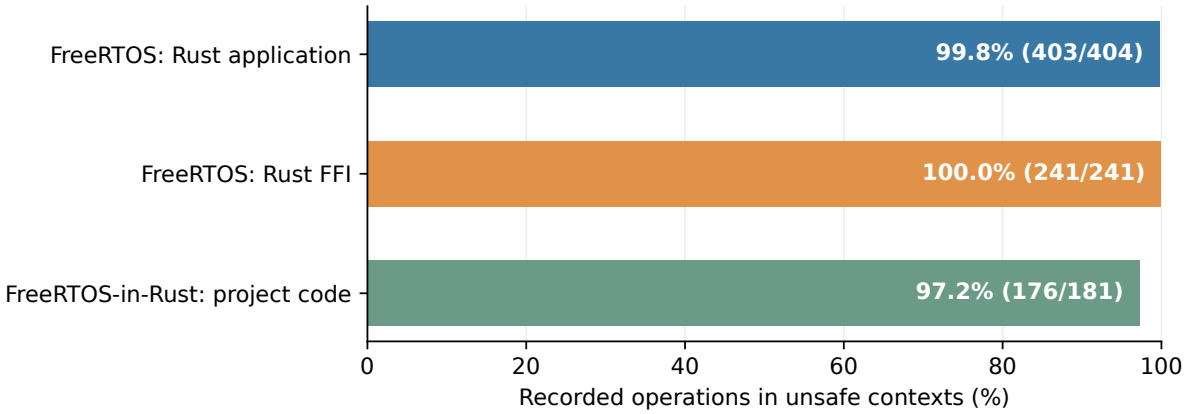


Fig. 11: Operations within unsafe contexts for conventional FreeRTOS Rust components and FreeRTOS-in-Rust project code. The analysed source scopes differ.

In FreeRTOS-in-Rust, 176 of 181 recorded operations (97.2%) occur within `unsafe` contexts. The analysis also identifies one resolved and 21 unresolved raw-pointer dereferences. Rivet similarly contains one resolved raw-pointer dereference, alongside 31 unresolved

records. However, these inventories cover only project-written application and board-support code, not the external Rust-native kernel implementations. Their results therefore provide additional observations about low-level code rather than a direct safety comparison between kernels.

Finally, the benchmark-specific source tracing connects these observations to the evaluated workloads. It covers 21–25 comparable cases per matched configuration and identifies 63–75 FFI symbols, depending on the RTOS. This provides information about which interfaces the benchmark code can reach, although the traces do not measure how often those calls execute.

6.4 Integration Complexity

The differences between C and Rust extend beyond performance, memory usage and safety. Their implementations also differ in how much code they require and how that code is organised. This section examines these differences through code size, function complexity and dependencies. The complete measurements are provided in Appendix E.

Fig. 12 compares the amount of hand-written benchmark and integration code across the evaluated configurations.

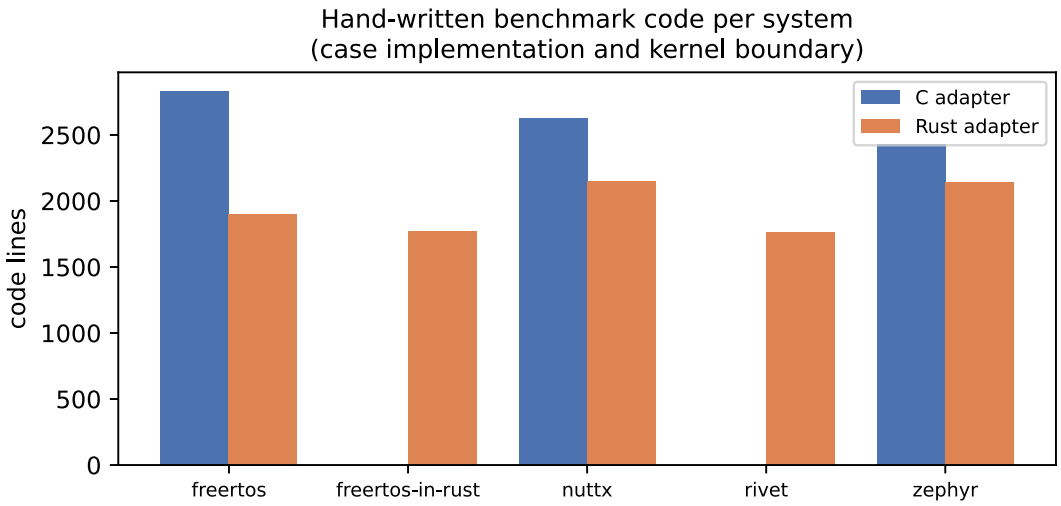


Fig. 12: Hand-written benchmark and kernel-integration code across the evaluated configurations.

Across the matched RTOSs, the Rust application and FFI modules contain fewer lines of hand-written code than the corresponding C implementations. The C configurations also include substantial kernel and SDK code, which is reported separately in Appendix E.

However, the shared benchmark runners show a different pattern. The C runner contains 493 lines of code across four functions, whereas the Rust runner contains 636 lines across

83 functions. The maximum cyclomatic complexity is 250 for C and 113 for Rust, while maximum cognitive complexity is 315 and 61, respectively. The Rust runner therefore has more functions but lower measured maximum complexity.

This difference also appears in the integration modules, although their complexity is generally lower than that of the shared runners. Maximum cyclomatic complexity ranges from 19 to 29 for the C ports and from 6 to 12 for the Rust FFI modules. The C ports also have 4–12 declared dependencies, compared with two for each Rust FFI module.

The two runners also differ in their source structure. They share 104 distinct identifiers. Dividing this intersection by the 279 identifiers in their union gives a Jaccard similarity of 0.37. This describes identifier overlap rather than functional equivalence. Likewise, the dependency counts reflect declared imports and includes, not all function calls or FFI interactions.

7 Discussion

The results show that differences between C and Rust depend on the workload, RTOS and hardware platform. Beyond the individual measurements, their implications must be considered when evaluating the suitability of either language for embedded RTOS applications. This chapter discusses the observed trade-offs in performance, memory usage, safety and integration complexity.

7.1 Performance and Resources

Across the matched configurations, C and Rust achieve similar execution times for most benchmark operations. Since both implementations use the same C-based kernel, much of their scheduling, synchronisation and interrupt behaviour is shared. The reported median operation times are often close for the matched C and Rust configurations. This descriptive comparison does not establish statistical equivalence, and some measurements use a shared C routine.

The communication benchmarks show where this conclusion has limits. Rust achieves lower payload-transfer latency on several FreeRTOS and Zephyr boards and higher IPC throughput on the FreeRTOS Arm configurations examined here; on ESP32-C6 the throughput difference is close to zero. These results are not contradictory. The payload-transfer benchmark measures the latency of individual transfers, whereas the throughput benchmark measures the amount of data transferred over time. Their results depend on different aspects of the implementation, including copying, buffer handling, RTOS behaviour and adapter calls. The results demonstrate workload-specific differences, but do not identify which implementation detail causes them or establish that one language is inherently faster.

Besides comparing C and Rust applications on the same kernel, the evaluation also includes the Rust-native RTOSs FreeRTOS-in-Rust and Rivet. On STM32F4, FreeRTOS-in-Rust records lower preemption latency than conventional FreeRTOS, but higher task-handoff and notification latencies. Rivet performs better in some communication benchmarks, while FreeRTOS-in-Rust performs better in others. Neither Rust-native RTOS is consistently faster. These differences reflect the complete kernel and application implementations rather than the programming language alone.

Memory consumption introduces a clearer trade-off. Although per-task overhead remains identical between matched C and Rust applications, Rust-linked firmware generally occupies more flash and sometimes more static RAM. This means that similar runtime

performance does not necessarily come with an equally small memory footprint. On memory-constrained microcontrollers, the additional flash or RAM requirements may limit which dependencies or application features can be included.

The difference between runtime and static memory is particularly relevant. The similar per-task overhead indicates that using Rust does not inherently increase the kernel's bookkeeping requirements when the same kernel is retained. The larger static footprint instead reflects differences in the linked application, integration code and dependencies. It should therefore be evaluated for each build rather than assumed to be a fixed overhead of the Rust language.

Overall, the results show that Rust is a practical option for applications on C-based RTOS kernels when execution time is the main concern. The more visible costs in the evaluated configurations are the additional firmware footprint and workload-specific performance differences, especially in communication. Whether these costs are acceptable depends on the timing requirements and available memory of the target system.

7.2 Safety at the Boundary

The safety analysis reveals an important difference in how C and Rust organise low-level operations. In C, pointer dereferences occur in both application and adapter code, while pointer arithmetic is mainly found in the applications. The analysed Rust applications, by contrast, contain no resolved raw-pointer operations. These operations instead appear in the FFI boundaries. This suggests that Rust can make direct pointer handling easier to locate and review, rather than eliminating the need for it.

However, this separation is not as straightforward as the pointer counts alone suggest. FreeRTOS stands out because almost all recorded Rust application operations occur within `unsafe` contexts, compared with roughly 70% on Zephyr and NuttX. At the same time, its FFI boundary contains considerably fewer resolved raw-pointer dereferences. One possible explanation is that the FreeRTOS application performs more RTOS interactions and global-state accesses directly within `unsafe` contexts, whereas the other implementations organise these operations differently. The inventory supports this distinction. FreeRTOS records both RTOS calls and global-state accesses in its Rust application, while the Zephyr and NuttX application inventories contain no recorded global-state accesses. Nevertheless, the different operation counts and code structures prevent drawing conclusions about their relative safety.

The FFI boundaries introduce further requirements. Their correctness depends on whether pointers remain valid and whether ownership and lifetime assumptions are respected across the language boundary. As discussed in the RustBelt research, a safe interface must be supported by a sound implementation. The Rust firmware also contains

C support code, while the underlying kernels and hardware-support components remain outside the compiler's safety checks. Keeping unsafe operations in dedicated modules can therefore make them easier to identify, but does not guarantee that the complete firmware is memory-safe.

The Rust-native results reinforce this limitation. FreeRTOS-in-Rust and Rivet also contain low-level operations in their analysed project code, although the external kernel implementations were not included in the inventories. These results do not establish whether Rust-native kernels are safer than C-based alternatives. Instead, they show that the need to examine low-level interfaces extends beyond C/Rust interoperability.

7.3 Integration Complexity

Besides performance and memory safety, the effort required to integrate and maintain an application is an important consideration when choosing between C and Rust. The results show differences in code size, organisation and integration requirements.

The Rust applications and their boundary modules contain fewer lines of hand-written code than the corresponding C implementations. However, Rust also requires interfaces to communicate with the C-based kernels, which must be updated when the kernel APIs change. A smaller implementation therefore does not necessarily mean less maintenance work.

The implementations also differ in how their logic is organised. The shared C runner handles much of its benchmark logic in a few large functions, while the Rust runner distributes it across more, smaller functions. This results in lower measured per-function complexity for Rust, although it does not necessarily make the code easier to maintain.

Another challenge is keeping the separate C and Rust benchmark runners consistent. Changes to workloads or measurement procedures must be applied to both implementations to ensure a fair comparison. This additional work is not reflected in the complexity measurements.

8 Conclusion

This thesis compared C and Rust applications on embedded real-time operating systems across four hardware platforms and five kernel implementations. The evaluation examined execution time, memory usage, safety-relevant source operations and integration complexity, while keeping the hardware and kernel unchanged for direct C/Rust comparisons.

The performance results show that C and Rust achieve similar execution times for most operations on the same RTOS and hardware. Larger differences occur mainly in communication benchmarks, where the results depend on the kernel and platform. The Rust-native kernels also show different strengths and weaknesses across operations. Neither comparison establishes a general performance advantage for one programming language.

Memory usage presents a clearer trade-off. Per-task memory overhead is identical between matched C and Rust implementations, while Rust-linked firmware generally requires more flash memory and sometimes more static RAM. These additional requirements may be relevant for microcontrollers with limited resources.

The analysis resolves no raw-pointer dereferences, arithmetic or conversions in the matched Rust application code, while resolving such operations in its FFI boundaries. One application raw-pointer record per NuttX configuration remains unresolved. This separates many low-level memory accesses from the application code, but does not eliminate the need to review interfaces to C kernels and hardware-support components. The analysis identifies these boundaries rather than establishing the memory safety of the complete firmware.

The integration results show that Rust requires fewer lines of hand-written application and boundary code than the corresponding C implementations. However, Rust introduces interfaces to the C-based kernels that must be maintained, and keeping separate C and Rust benchmark implementations consistent introduces additional work. The measured complexity differences therefore do not establish that either language is easier to maintain.

Beyond these findings, the thesis provides a documented benchmarking procedure with firmware configurations, build settings and archived measurements. The archived primary campaign supports checking the matched C/Rust results and extending the comparison. The supplementary raw bundle for the Rust-native timing results is unavailable, so those retained summaries cannot currently be reproduced from raw samples.

Overall, Rust is a viable alternative to C for the investigated RTOS applications, with similar execution times in most benchmarks and fewer directly expressed raw-pointer operations in application code. Its additional firmware footprint and integration requirements remain important considerations when choosing a language for a particular embedded system.

8.1 Limitations

The findings are limited to the evaluated hardware platforms, RTOS implementations, build configurations and benchmark workloads. Only one physical device of each board type was tested, and measurements were collected in a fixed order. The results therefore do not capture variation between devices or possible effects of measurement order.

The benchmarks measure individual RTOS operations rather than complete applications. Although C and Rust follow the same benchmark definitions on matched hardware and kernels, their implementation and adapter paths can differ. Some workloads also vary between RTOSs, particularly the loaded-jitter benchmark. Consequently, comparisons across different kernels require additional care. The measured execution times and percentiles do not establish worst-case latency bounds or deadline guarantees.

The safety analysis covers selected source operations and project-written integration code, but not the internal implementations of third-party kernels, SDKs or HALs. Its operation counts cannot establish whether memory-safety defects are present. Similarly, the integration-complexity measurements describe code structure rather than actual development or maintenance effort.

8.2 Future Work

Future work could extend the evaluation to additional hardware platforms, RTOSs and more representative embedded applications. Measuring complete applications with interacting tasks, communication and memory management would help determine whether the differences observed in individual benchmarks also affect larger systems.

The measurement campaign could also include several physical devices of each board type and a randomised or counterbalanced execution order. More detailed profiling of selected benchmarks could help identify the causes of the larger C/Rust performance differences, particularly in communication operations.

Finally, a unified static-analysis tool could apply consistent criteria to both languages, including pointer validity, ownership, lifetimes and FFI-boundary operations. Combining analysis with targeted testing would identify potential memory-safety defects rather than only counting safety-relevant operations.

Bibliography

[1] D. Song *et al.*, "SoK: Sanitizing for Security," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1275–1295. doi: 10.1109/SP.2019.00010.

[2] Office of the National Cyber Director, "Back to the Building Blocks: A Path Toward Secure and Measurable Software," technical report, Feb. 2024. Accessed: Oct. 10, 2026. [Online]. Available: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf

[3] M. Salehi, L. Degani, M. Roveri, D. Hughes, and B. Crispo, "Discovery and Identification of Memory Corruption Vulnerabilities on Bare-Metal Embedded Devices," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1124–1138, 2023, doi: 10.1109/TDSC.2022.3149371.

[4] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A Large-Scale Analysis of the Security of Embedded Firmwares," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 95–110. [Online]. Available: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/costin

[5] M. S. H. Shaon and M. S. Akter, "Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models," *Electronics*, vol. 14, no. 22, p. 4449, 2025, doi: 10.3390/electronics14224449.

[6] T. Vandervelden, R. D. Smet, D. Deac, K. Steenhaut, and A. Braeken, "Overview of Embedded Rust Operating Systems and Frameworks," *Sensors*, vol. 24, no. 17, p. 5818, 2024, doi: 10.3390/s24175818.

[7] The Rust Project, "The Rust Programming Language: What Is Ownership?." Accessed: Sep. 19, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html

[8] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry, "Rust for Embedded Systems: Current State and Open Problems," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2296–2310. doi: 10.1145/3658644.3690275.

[9] J. W. S. Liu, *Real-Time Systems*. Upper Saddle River, NJ: Prentice Hall, 2000.

[10] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Springer Nature Switzerland, 2024. doi: 10.1007/978-3-031-45410-3.

[11] L. Sha, R. Rajkumar, and J. P. Lehoczky, “Priority Inheritance Protocols: An Approach to Real-Time Synchronization,” *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990, doi: 10.1109/12.57058.

[12] FreeRTOS, “FreeRTOS Kernel: Queue Management API.” Accessed: Oct. 20, 2026. [Online]. Available: https://github.com/FreeRTOS/FreeRTOS-Kernel/blob/main/include/queue.h

[13] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, “Dynamic Storage Allocation: A Survey and Critical Review,” in *Memory Management, Lecture Notes in Computer Science*, vol. 986, Springer, 1995, pp. 1–116. doi: 10.1007/3-540-60368-9_19.

[14] The Rust Project, “The Rust Programming Language: Unsafe Rust.” Accessed: Sep. 19, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html

[15] The Rust Project, “The Rust Reference.” Accessed: Oct. 10, 2026. [Online]. Available: https://doc.rust-lang.org/reference/

[16] “Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.” [Online]. Available: https://www.iso.org/standard/78176.html

[17] T. J. McCabe, “A Complexity Measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976, doi: 10.1109/TSE.1976.233837.

[18] G. A. Campbell, “Cognitive Complexity: An Overview and Evaluation,” in *Proceedings of the 2018 International Conference on Technical Debt*, ACM, 2018, pp. 57–58. doi: 10.1145/3194164.3194186.

[19] L. C. Briand, J. W. Daly, and J. Wüst, “A Unified Framework for Coupling Measurement in Object-Oriented Systems,” *IEEE Transactions on Software Engineering*, vol. 25, no. 1, pp. 91–121, 1999, doi: 10.1109/32.748920.

[20] R. P. Kar and K. Porter, “Rhealstone: A Real-Time Benchmarking Proposal,” *Dr. Dobb's Journal*, 1989, Accessed: Oct. 20, 2026. [Online]. Available: https://jacobfilipp.com/DrDobbs/articles/DDJ/1989/8902/8902a/8902a.htm

[21] R. P. Kar, “Implementing the Rhealstone Real-Time Benchmark,” *Dr. Dobb's Journal*, Apr. 1990, Accessed: Oct. 11, 2026. [Online]. Available: https://jacobfilipp.com/DrDobbs/articles/DDJ/1990/9004/9004d/9004d.htm

[22] Express Logic, Inc., “Thread-Metric RTOS Test Suite: README.” Accessed: Oct. 20, 2026. [Online]. Available: https://github.com/eclipse-threadx/threadx/blob/master/utility/benchmarks/thread_metric/thread_metric_readme.txt

[23] RTOSBench Project, “RTOSBench: A Benchmark for Real-Time Operating Systems.” Accessed: Oct. 20, 2026. [Online]. Available: https://github.com/gchamp20/RTOSBench

[24] R. R. Belleza and E. P. de Freitas, “Performance Study of Real-Time Operating Systems for Internet of Things Devices,” *IET Software*, vol. 12, no. 3, pp. 176–182, 2018, doi: 10.1049/iet-sen.2017.0048.

[25] G. Wang, “Concurrency Frameworks for Embedded Systems: A Comparative Evaluation of FreeRTOS, RTIC, and Embassy,” *IET Computers & Digital Techniques*, 2026, doi: 10.1049/cdt2/7881841.

[26] A. Levy *et al.*, “Multiprogramming a 64 kB Computer Safely and Efficiently,” in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017. doi: 10.1145/3132747.3132786.

[27] I. Culic, A. Vochescu, and A. Radovici, “A Low-Latency Optimization of a Rust-Based Secure Operating System for Embedded Devices,” *Sensors*, vol. 22, no. 22, p. 8700, 2022, doi: 10.3390/s22228700.

[28] E. Frank, K. Schleiser, R. Fouquet, K. Zandberg, C. Amsüss, and E. Baccelli, “Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers,” in *Proceedings of the IEEE International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, 2025, pp. 241–245. doi: 10.1109/DCOSS-IoT65416.2025.00040.

[29] I. Plauska, A. Liutkevičius, and A. Janavičiūtė, “Performance Evaluation of C/C++, MicroPython, Rust and TinyGo Programming Languages on ESP32 Microcontroller,” *Electronics*, vol. 12, no. 1, p. 143, 2023, doi: 10.3390/electronics12010143.

[30] I. Embuena *et al.*, “Comparison of Platforms and Languages for Performance and Power in Embedded Systems,” *Electronics*, vol. 15, no. 16, p. 3682, 2026, doi: 10.3390/electronics15163682.

[31] B. Thapa *et al.*, “Embedded Rust or C Firmware? Lessons from an Industrial Microcontroller Use Case with Ariel OS.” [Online]. Available: https://arxiv.org/abs/2604.25679

[32] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the Foundations of the Rust Programming Language,” in *Principles of Programming Languages (POPL)*, 2018. [Online]. Available: https://plv.mpi-sws.org/rustbelt/popl18/

[33] S. Höltervennhoff, P. Klostermeyer, N. Wöhler, Y. Acar, and S. Fahl, “``I wouldn't want my unsafe code to run my pacemaker'': An Interview Study on the Use, Comprehension, and Perceived Risks of Unsafe Rust,” in *32nd USENIX Security Symposium*

(USENIX Security 23), 2023, pp. 4277–4294. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity23-holtervennhoff.pdf

[34] O. K. A. Santoso, C. Kwee, and W. Chua, "Rust's Memory Safety Model: An Evaluation of Its Effectiveness in Preventing Common Vulnerabilities," *Procedia Computer Science*, vol. 227, pp. 119–127, 2023, doi: 10.1016/j.procs.2023.10.509.

[35] C. Zhang, Y. Feng, Y. Zhang, Y. Dai, and B. Xu, "Beyond Memory Safety: an Empirical Study on Bugs and Fixes of Rust Programs," in *Proceedings of the IEEE International Conference on Software Quality, Reliability and Security (QRS)*, 2024, pp. 272–283. doi: 10.1109/QRS62785.2024.00035.

[36] I. McCormack, J. Sunshine, and J. Aldrich, "A Study of Undefined Behavior Across Foreign Function Boundaries in Rust Libraries." [Online]. Available: https://arxiv.org/abs/2404.11671

[37] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu, "An Empirical Study of Rust-for-Linux: The Success, Dissatisfaction, and Compromise," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 425–443. [Online]. Available: https://www.usenix.org/system/files/atc24-li-hongyu.pdf

[38] M. Munch, M. Lindner, J. Eriksson, P. Dzialo, and P. Lindgren, "Safety and Security Critical Systems with Rust," *Journal of Signal Processing Systems*, vol. 98, no. 2, 2026, doi: 10.1007/s11265-026-02011-x.

[39] O. Braunsdorf, T. Lange, K. Hohentanner, J. Horsch, and J. Kinder, "SafeFFI: Efficient Sanitization at the Boundary Between Safe and Unsafe Code in Rust and Mixed-Language Applications," in *35th USENIX Security Symposium (USENIX Security 26)*, USENIX Association, 2026, pp. 2843–2861. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/braunsdorf

[40] R. Jung, B. Kimock, C. Poveda, E. S. Muñoz, O. Scherer, and Q. Wang, "Miri: Practical Undefined Behavior Detection for Rust," *Proceedings of the ACM on Programming Languages*, vol. 10, no. POPL, pp. 1383–1411, 2026, doi: 10.1145/3776690.

[41] H.-M. Chen, B. Lu, X. He, X. Zhang, and K. Sun, "CULPA: Universal Detection of Memory-Safety Bugs in Unsafe Rust Through the Lens of Safety Requirements," in *Proceedings of the 35th USENIX Security Symposium (USENIX Security 26)*, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/chen-hung-mao

[42] W. Li, W. Chen, and J. Xue, "From Raw Pointers to Memory Safety: A Modular Demand-Driven Typestate Analysis for Rust," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 1817–1845, 2026, doi: 10.1145/3798266.

[43] Lobaro, “FreeRTOS-rust: Rust Crate for FreeRTOS.” Accessed: Sep. 19, 2026. [Online]. Available: https://github.com/lobaro/FreeRTOS-rust

[44] The Zephyr Project, “Zephyr Documentation: Rust Language Support.” Accessed: Sep. 19, 2026. [Online]. Available: https://docs.zephyrproject.org/latest/develop/languages/rust/index.html

[45] The Apache NuttX Project, “Rust in NuttX.” Accessed: Oct. 20, 2026. [Online]. Available: https://nuttx.apache.org/docs/latest/guides/languages/rust.html

[46] N. Yoshimura, H. Oyama, and T. Azumi, “TECS/Rust: Memory-safe Component Framework for Embedded Systems,” in *Proceedings of the 27th IEEE International Symposium on Real-Time Distributed Computing (ISORC)*, 2024. doi: 10.1109/ISORC61049.2024.10551370.

[47] N. Yoshimura, H. Oyama, and T. Azumi, “TECS/Rust-OE: Optimizing Exclusive Control in Rust-Based Component Systems for Embedded Devices,” in *Proceedings of the 28th IEEE International Symposium on Real-Time Distributed Computing (ISORC)*, 2025. doi: 10.1109/ISORC65339.2025.00051.

[48] N. Yoshimura, H. Oyama, and T. Azumi, “FMP3+TECS/Rust: Memory-Safe Component Framework for Multiprocessor Embedded Systems,” *IEEE Open Journal of the Industrial Electronics Society*, 2026, doi: 10.1109/OJIES.2026.3666358.

[49] S. Motogna, A. Vescan, and C. Șerban, “Empirical investigation in embedded systems: Quality attributes in general, maintainability in particular,” *Journal of Systems and Software*, vol. 201, p. 111678, 2023, doi: 10.1016/j.jss.2023.111678.

[50] STMicroelectronics, “STM32F4DISCOVERY: STM32F4 Discovery kit with STM32F407VG MCU.” Accessed: Oct. 08, 2026. [Online]. Available: https://www.st.com/en/evaluation-tools/stm32f4discovery.html

[51] The Zephyr Project, “FK723M1-ZGT6 board documentation (Fanke STM32H723ZGT6 core board).” Accessed: Oct. 08, 2026. [Online]. Available: https://docs.zephyrproject.org/latest/boards/fanke/fk723m1_zgt6/doc/index.html

[52] Espressif Systems, “ESP32-C6-DevKitC-1: ESP-IDF hardware reference.” Accessed: Oct. 08, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/hw-reference/esp32c6/user-guide-devkitc-1.html

[53] Nordic Semiconductor, “nRF52840 DK (PCA10056).” Accessed: Oct. 08, 2026. [Online]. Available: https://www.nordicsemi.com/Products/Development-hardware/nRF52840-DK

[54] Mozilla, “rust-code-analysis Documentation: Supported Metrics and Supported Languages.” Accessed: Oct. 20, 2026. [Online]. Available: https://mozilla.github.io/rust-code-analysis/

List of Abbreviations

API Application Programming Interface
AST Abstract Syntax Tree
CDC Communications Device Class
CPU Central Processing Unit
DSP Digital Signal Processing
DWT Data Watchpoint and Trace
ELF Executable and Linkable Format
FFI Foreign Function Interface
HAL Hardware Abstraction Layer
IPC Inter-Process Communication
ISR Interrupt Service Routine
JSON JavaScript Object Notation
kLOC thousand Lines of Code
LOC Lines of Code
LTO Link-Time Optimisation
POSIX Portable Operating System Interface
RAM Random Access Memory
RTOS Real-Time Operating System
SDK Software Development Kit
SoC System on a Chip
SRAM Static Random Access Memory
SWD Serial Wire Debug
UART Universal Asynchronous Receiver-Transmitter
USB Universal Serial Bus

Documentation of AI Assistance

This thesis was prepared with assistance from several AI tools, used for the distinct purposes recorded below. AI tools assisted with drafting and revising the text as well as troubleshooting and analysis. The author directed the research, reviewed the wording, sources, calculations and interpretation in every case, and takes responsibility for the final text. No result reported in this thesis was produced by an AI system.

Tool / model	Purpose	Scope and evidence
Codex	Architecture outline and source-code analysis.	Initial architecture outline of the accompanying RTOS benchmark project, and later inspection of the C and Rust implementation sources against the thesis claims with targeted corrections. The author directed the work and reviewed all output.
DeepSeek v4.1 Pro	Drafting, revision and software troubleshooting.	Diagnosis of software and build failures, and drafting and review of thesis text.
DeepSeek Flash	Drafting, revision and software troubleshooting.	Software and build troubleshooting, and drafting and review of thesis text.
ChatGPT	Drafting and revision of thesis text.	Drafting and editing thesis chapters, including the conclusion, with the author reviewing the output.
Local models (Qwen3.8 27B, Devstral Small 2 24B, Qwen3-Coder 30B-A3B, Qwen3-Coder-Next; Q4_K_M)	Comparing the project's implementations.	Local inference over the C and Rust implementations, on the author's own machine, for implementation comparison and cross-checking.

Table VI: AI tools used, their purpose, and the scope of their use.

Local models ran on the author's machine. Other tools used hosted services. Quantitative results derive from hardware measurements, compiled-binary analysis, source-code analysis and statistical processing, as described in the Methodology chapter. Quantitative figures come from the project's analysis scripts. The author reviewed the results and interpretation.

A Benchmark Specifications

This appendix records the workload, measurement boundary and available checks for the 26 benchmark cases listed in Table V. Case numbers follow the thesis table and differ from the zero-based protocol identifiers. Thesis cases 1–26 map respectively to protocol IDs 0, 1, 12, 13, 14, 2, 5, 6, 8, 16, 17, 7, 18, 19, 27, 4, 10, 20, 21, 25, 3, 11, 15, 22, 23 and 24. Protocol IDs 9, 26 and 28 are intentionally excluded. Timing values are collected in timer ticks unless a different unit is stated. Resources are generally created before a timed operation and released afterwards. Differences mentioned here matter when interpreting measurements across kernels or languages.

Some benchmarks use C measurement or control routines in both the C and Rust configurations. These include interrupt and periodic-timing measurements and selected CPU-load and heap-fragmentation cases. Other benchmarks, such as the payload sweep on Zephyr and NuttX, delegate part of the transfer operation from Rust to C. Consequently, results for these cases include shared C execution and should not be interpreted as comparisons of entirely independent C and Rust workload implementations. These descriptions refer to the firmware revision recorded with the archived measurements; later working-tree revisions are not substituted for that evidence.

1. `timer_overhead` — Two timestamp readings bracket the runner's case-dispatch path. The result includes the timer-access calls and dispatch instructions between them, but not record transmission. Since C and Rust use different dispatch code, this is a measurement-path baseline rather than the isolated cost or resolution of the timer.

2. `queue_roundtrip` — A single task sends a 32-bit value to a capacity-one queue and receives it back. Timing begins before the send and ends after the receive. The returned value must match the sent value. Queue creation and destruction lie outside the timed interval; no other application task is involved.

3. `semaphore_latency` — A single task gives an initially empty binary semaphore and immediately takes it again. Timing covers both operations. As the semaphore is available when taken, no blocking or task hand-off is intended. Successful acquisition is required.

4. `mutex_uncontended` — A single task locks and unlocks a free mutex. Both operations lie between the two timestamps; resource creation does not. No competing task holds the mutex, so priority inheritance and blocking are not exercised.

5. `alloc_latency` — One 64-byte block is allocated and then freed, with both operations included in the timed interval. The returned allocation must succeed. The case uses the kernel-provided allocator where available and is not supported by every Rust-native kernel. Compiler barriers and FFI paths differ between some C and Rust adapters.

6. `task_handoff` — A waiting peer task is released, and timing ends when that peer records its wake timestamp. The interval includes the release operation and the scheduler's transition to the receiver. In the primary FreeRTOS, Zephyr and NuttX ports, the receiver has higher priority and preempts the sender. The Rust-native kernels use different arrangements. The recorded sample does not independently store task priorities or a preemption flag.

7. `yield_roundtrip` — The benchmark task yields to an equal-priority peer and measures the interval until it resumes. The two-task configuration is the baseline. Supported C-kernel ports also use 8, 16 and 32 peers; these are distinct variants, not additional samples of the two-task workload. The implementation checks that a peer ran and may retry the yield up to a fixed limit. The Rust-native kernels use different cooperative release or yielding mechanisms.

8. `preempt_latency` — A higher-priority task waits for a release while a lower-priority task executes. The first timestamp is taken immediately before the release, and the second by the high-priority task after waking. The primary C-kernel implementations release a semaphore waiter; FreeRTOS-in-Rust uses an interrupt-driven release instead. Available sequence checks confirm that the receiver ran.

9. `mutex_priority_inheritance` — A low-priority task holds a mutex, a high-priority task waits to acquire it, and a medium-priority task performs a bounded spin of approximately 2 ms. Timing runs from the high-priority task's lock attempt to successful acquisition. The medium-priority work makes the effect of priority inheritance observable, but there is no automatic pass/fail threshold based on the measured interval.

10. `scheduler_overhead` — The benchmark invokes the scheduler when no alternative application task is ready. Timing covers the invocation and return without an intended context switch. Implementations may reject samples interrupted by a system tick and retry. Ports without a suitable switch-free scheduler path do not provide the case.

11. `notification_latency` — A peer signals the waiting benchmark task. The measured interval starts at the peer's signal and ends when the benchmark task resumes. FreeRTOS uses direct task notification; Zephyr and NuttX use semaphore-based alternatives, and FreeRTOS-in-Rust uses an event-group mechanism. These primitives are functionally similar for this workload but do not have identical implementation costs.

12. `ipc_pingpong` — Two capacity-one queues connect a benchmark task and an echo peer. The benchmark sends a 32-bit request, the peer returns it unchanged, and timing stops when the reply is received. The reply is checked against the request. Queue operations, waiting and task scheduling are part of the timed round trip.

13. `ipc_payload_sweep` — The same request-and-reply pattern is repeated with payload sizes of 1, 4, 16, 64 and 256 bytes. Each size uses its corresponding queues and echo task. Timing includes the transfers and their copying costs, but every returned byte is checked after the end timestamp. Samples are associated with the relevant payload-size variant. Some Zephyr and NuttX Rust paths delegate the transfer to C adapter routines.

14. `ipc_throughput` — A producer sends 16,384 messages of 64 bytes, totalling 1 MiB, through a queue of depth eight. A consumer drains the queue. The measured interval covers the transfer and, on the primary C-kernel paths, consumer completion. The result is converted from the transfer duration into bytes per second. Message counts are checked, but payload bytes are not individually verified. FreeRTOS-in-Rust has a different end condition, so its final queue-drain time may not be included.

15. `zero_copy_ipc` — Two forms of a 256-byte request-and-reply transfer are timed: a copied payload and a reference to shared storage. The reported result is copied time minus reference-transfer time. The returned pointer and payload contents are checked. A non-positive difference is recorded as no observed benefit; its count is retained separately and the sentinel is excluded from positive-saving percentiles. A positive-only distribution must not be interpreted as including all observations.

16. `isr_to_task` — A timer interrupt timestamps handler entry and signals a waiting task. Timing ends when the task resumes after the wait, covering the interrupt-to-task wake-up path. On the paired FreeRTOS, Zephyr and NuttX configurations, the corrected Rust adapter calls the same C measurement routine used by the C runner; the FFI call surrounds rather than interrupts the timed operation. Rust-native kernels have their own timer and signalling arrangements.

17. `trigger_to_isr` — A task records a timestamp, requests a dedicated software-triggered interrupt and compares that timestamp with handler entry. Primary FreeRTOS, Zephyr and NuttX ports mask interrupts before recording the first timestamp, pend the interrupt and then unmask. The Rust-native kernels omit this masking sequence. This measures the implemented software-request-to-entry path, including request and unmasking effects where applicable, rather than a pure hardware exception-entry constant.

18. `nested_irq_overhead` — A lower-priority interrupt handler executes a fixed spin while a higher-priority interrupt is requested ten times. A separate baseline runs without the

higher-priority interruptions. The result is the added execution time compared with that baseline. The disabling control is a separate compile-time or manual check rather than a control automatically recorded in every campaign run. Some ports select a minimum from repeated pairs.

19. `tick_overhead` — Available trace hooks capture timestamps around the kernel's periodic tick-processing path. Each sample reports the time attributed to one tick handler, without a deliberately included later application task switch. Hook locations are port-dependent, and kernels without suitable hooks do not report this case.

20. `max_interrupt_disabled` — A timer is configured to interrupt after approximately 50 µs at a priority subject to kernel masking. The reported interval is the observed handler-entry time minus the nominal probe delay. It includes the direct timer-arming sequence present in the port's measurement definition. It is an interrupt-deferral observation, not the full length of a critical section and not a guaranteed maximum. On corrected paired FreeRTOS, Zephyr and NuttX paths, both languages use the same C measurement routine. Extended masking was checked separately rather than within each ordinary run.

21. `periodic_jitter` — The task wakes according to a nominal 1 ms periodic delay. The measured wake-to-wake interval is compared with that nominal period. Because the metric uses consecutive wake intervals, a constant release delay cancels; the result instead captures changes in timing and any persistent period offset. Large deviations are retained rather than automatically treated as missed deadlines.

22. `periodic_jitter_loaded` — A hardware-timer release schedule has a nominal period of 750 µs. Each valid result is the observed activation time minus its expected position on the release grid, so persistent lateness remains visible. Load generation varies by port. STM32F4 and STM32H723 FreeRTOS use settings 0, 100, 150 and 200 to adjust a high-priority interrupt-masked work interval; the settings are not CPU utilisation values. Other boards and kernels use fixed or different sources of load. Missed or coalesced releases produce a sentinel, which is counted separately and omitted from latency percentiles. The release index is realigned after a miss. On corrected paired C-kernel paths, the C and Rust runners call the same C timing function.

23. `sleep_accuracy` — A task requests an approximately 1 ms delay and measures the elapsed time until it resumes. The result is the absolute difference between the measured duration and the nominal request. FreeRTOS uses a one-tick delay, while Zephyr and NuttX use millisecond-based APIs with their respective tick or scheduler semantics. The corrected Zephyr Rust implementation matches the C `k_msleep(1)` conversion and sleep operation; timestamp access may still differ through FFI.

24. `idle_cpu_overhead` — Over a 500 ms idle observation window, trace hooks accumulate the cycles spent handling periodic kernel ticks. The result is this total divided by the window duration, multiplied by one million and reported in ppm. Other kernel activity is not comprehensively counted, so the metric is tick-related CPU share rather than total RTOS overhead. Availability depends on port-specific accounting support.

25. `cpu_overhead_under_load` — A synthetic worker repeats a 10 ms period with nominal busy fractions of 25, 50, 75 and 100 %. It spins for the configured portion and sleeps for the remainder. Each variant measures non-idle CPU share over a 500 ms window, expressed in ppm. The result includes application work as well as kernel activity. The worker does not adjust its spin duration using feedback from the observed sleep time, so the configured fraction is an offered workload rather than an exact measured utilisation.

26. `heap_fragmentation` — The allocator is filled with 256-byte blocks down to a port-specific remaining-free-space margin. Every other block is freed to create separated gaps. The metric is one million times one minus the ratio of the largest available free block to the total free heap; it is reported in ppm. FreeRTOS uses a fixed 16 KiB margin, Zephyr generally uses one eighth of the heap with a minimum margin, and NuttX uses a 20 % margin. FreeRTOS and Zephyr probe allocations to estimate the largest block, while NuttX reads allocator statistics. These differences prevent the metric from representing a strictly identical heap state on all kernels. The defragmented control was checked separately, not as part of every recorded run.

B Complete Benchmark Results

Each benchmark cell gives the pooled median above and p99 below. A dot marks a case the configuration does not advertise; a single value means p99 is unavailable. The suffixes k and M denote 10^3 and 10^6. Parameterised rows show selected variants: the base peer count for case 7, 256 bytes for case 13, the highest recorded load for case 22 and 100 % offered load for case 25. For latency, lower is better; for IPC throughput (14), higher is better and p99 is the higher-throughput tail. An asterisk marks a configuration with filtered observations, listed in the note table after the five value tables.

Path	Advertised cases (at least one measured run)
FreeRTOS STM32F4 C	1–26
FreeRTOS STM32F4 Rust	1–26
FreeRTOS STM32H723 C	1–26
FreeRTOS STM32H723 Rust	1–26
FreeRTOS ESP32-C6 C	1–18, 20–23
FreeRTOS ESP32-C6 Rust	1–18, 20–23
FreeRTOS nRF52840 C	1–23, 26
FreeRTOS nRF52840 Rust	1–23, 26
NuttX STM32F4 C	1–18, 20–23, 26
NuttX STM32F4 Rust	1–18, 20–23, 26
NuttX STM32H723 C	1–18, 20–23, 26
NuttX STM32H723 Rust	1–18, 20–23, 26
NuttX ESP32-C6 C	1–18, 20–23, 26
NuttX ESP32-C6 Rust	1–18, 20–23, 26
NuttX nRF52840 C	1–18, 20–23, 26
NuttX nRF52840 Rust	1–18, 20–23, 26
Zephyr STM32F4 C	1–18, 20–23, 26
Zephyr STM32F4 Rust	1–18, 20–23, 26
Zephyr STM32H723 C	1–18, 20–23, 26
Zephyr STM32H723 Rust	1–18, 20–23, 26
Zephyr ESP32-C6 C	1–18, 20–23, 26
Zephyr ESP32-C6 Rust	1–18, 20–23, 26
Zephyr nRF52840 C	1–18, 20–23, 26
Zephyr nRF52840 Rust	1–18, 20–23, 26
FreeRTOS-in-Rust STM32F4 Rust	1–26
FreeRTOS-in-Rust STM32H723 Rust	1–26
Rivet RTOS STM32F4 Rust	1–4, 6–25

Table VII: Case coverage by board, RTOS and language.

Zephyr nRF52840 throughput uses 980 C and 987 Rust valid historical observations; 20 C and 13 Rust samples with zero or wrapped durations are excluded and counted in the table. These filtered observations do not support lower-tail or worst-case throughput claims. For the Rust-native kernels, only a summary-derived median is shown because the supplementary raw bundle is unavailable; p99 throughput is not reported.

Benchmark	Unit	STM32F4		STM32H723		ESP32-C6		nRF52840	
		C	Rust	C	Rust	C	Rust	C	Rust
Basic API									
1 Timer and runner overhead	µs	0.137	0.131	0.227	0.208	0.938	0.625	0.703	0.875
		0.137	0.131	0.227	0.235	1	0.625	0.703	0.875
2 Queue send and receive	µs	3.59	4.59	3.94	3.9	3.56	3.62	9.91	11.2
		3.59	4.59	3.95	3.93	3.56	3.69	12.4	13.6
3 Semaphore latency	µs	2.21	2.35	2.2	2.08	2.62	2.69	6.27	6.2
		2.21	2.35	2.23	2.12	2.69	2.75	6.27	6.2
4 Mutex uncontended	µs	3.08	3.27	2.89	2.76	3.06	3.06	7.73	7.8
		3.08	3.27	2.89	2.78	3.06	3.12	7.73	7.8
5 Allocation latency	µs	4.51	5.02	3.52	3.54	10.9	11	11.8	11.7
		4.54	5.04	3.73	3.54	11	18.3	11.8	11.7
Scheduling									
6 Task release to receiver	µs	5.4	5.46	4.69	4.42	7.38	7.44	12.1	11.1
		5.57	5.46	4.72	4.45	7.44	7.44	14.5	11.1
7 Yield round trip	µs	5.62	5.17	5.47	5.22	14.4	14.1	12.5	11.3
		5.62	5.17	5.47	5.22	16.7	14.1	12.5	11.3
8 Preemption latency	µs	5.14	5.12	4.45	4.28	7.12	7.25	11.3	10.4
		5.14	5.13	4.45	4.28	7.19	7.25	13.7	12.7
9 Priority inheritance	µs	18.2	17.9	16.9	16	22.6	22.6	45.2	41.6
		19.6	19.2	17.8	16.1	29.9	29.9	48.3	44.7
10 Scheduler overhead	µs	1.86	1.95	1.91	1.63	4.88	5	4.56	3.95
		1.86	1.95	2.02	1.85	4.94	5.06	4.56	3.95
11 Task notification	µs	3.67	3.77	3.74	3.8	6.62	6.75	9.45	8.97
		3.67	4.09	3.75	3.81	6.69	6.75	9.45	8.97
IPC									
12 IPC ping-pong	µs	17	18.2	16.5	16.4	21.4	21.4	42.9	42.3
		18.6	19.8	17.6	17.2	28.7	28.8	46	44.9
13 Payload sweep	µs	53.5	26.8	74.8	23.1	25.6	25.6	186	74.1
		55.1	26.8	75.7	23.1	33.1	32.9	188	76.6
14 IPC throughput	B/s	3.47M	3.92M	3.17M	4.57M	3.26M	3.24M	1.22M	1.7M
		3.47M	3.92M	3.17M	4.57M	3.26M	3.24M	1.22M	1.7M
15 Zero-copy IPC	ticks	6.11k	1.47k	30.3k	3.13k	66	65	9.07k	1.97k
		6.37k	1.73k	30.8k	3.65k	183*	182*	9.27k	2.16k

Table VIII: FreeRTOS benchmark results by board and language

Benchmark	Unit	STM32F4		STM32H723		ESP32-C6		nRF52840	
		C	Rust	C	Rust	C	Rust	C	Rust
Interrupts									
16 Interrupt handler to task	µs	4.8	4.57	4.01	3.8	5.88	5.88	11.4	9.8
		5.11	4.57	4.01	3.8	13.2	13.2	13.7	10.4
17 Trigger to interrupt	µs	0.214	0.208	0.177	0.15	1.62	1.62	0.594	0.625
		0.214	0.208	0.177	0.15	1.69	1.62	0.594	0.625
18 Nested interrupt	µs	61.9	61.9	10.5	17.4	45.9	45.9	100	101
		62	61.9	10.5	17.4	45.9	45.9	100	101
19 Tick overhead	µs	1.85	1.92	1.29	1.28	·	·	3.73	3.64
		1.85	1.92	1.29	1.28			3.73	3.64
20 Observed interrupt deferral	µs	0.536	0.577	0.437	0.463	6.31	6.38	1.53	1.53
		0.542	0.577	0.463	0.463	10.3	10.1	2.02	1.53
Timing & Sleep									
21 Periodic jitter	µs	0	0.00595	0.0538	0.00385	0	0	0	0.0156
		0	0.0119	0.0615	0.0115	0	0.000625	0.0156	0.0156
22 Loaded periodic jitter	µs	4.86	4.63	4.11	3.94	7	7	10.8	9.88
		4.86*	4.63*	4.13*	3.95*	54	56	60.5	61.9
23 Sleep accuracy	µs	2.47	3.41	1.52	2.23	2.44	2.69	5.34	7.53
		2.48	3.42	1.7	2.25	2.44	2.69	5.39	7.55
Load									
24 Tick-related CPU share while idle	ppm	568	568	517	440	·	·	·	·
		568	568	517	440				
25 CPU share under load	ppm	1M	1M	1M	1M	·	·	·	·
		1M	1M	1M	1M				
Memory									
26 Heap fragmentation	ppm	674k	674k	674k	674k	·	·	674k	674k
		674k	674k	674k	674k			674k	674k

FreeRTOS benchmark results by board and language (continued)

Benchmark	Unit	STM32F4		STM32H723		ESP32-C6		nRF52840	
		C	Rust	C	Rust	C	Rust	C	Rust
Basic API									
1 Timer and runner overhead	µs	0.232	0.143	0.0635	0.0404	0.175	0.163	0.375	0.578
		0.256	0.173	0.0635	0.0404	0.175	0.169	0.375	0.578
2 Queue send and receive	µs	8.58	8.13	2.39	2.22	5.67	5.58	17.4	16.5
		13.1	12.7	3.5	3.33	15.1	14.9	35	34.1
3 Semaphore latency	µs	0.738	0.762	0.335	0.337	0.706	0.825	1.81	2.44
		1.16	1.32	0.352	0.398	10.1	10.2	1.81	2.44
4 Mutex uncontended	µs	1.62	1.18	0.438	0.465	1.16	1.28	3.17	3.75
		1.98	5.78	0.477	0.492	10.5	10.6	3.17	3.75
5 Allocation latency	µs	4.38	4.48	1.23	1.24	4.12	4.26	9.06	9.06
		9.09	9.21	2.27	1.27	13.5	13.6	9.29	9.07
Scheduling									
6 Task release to receiver	µs	5.67	5.54	1.47	1.42	3.01	3.01	9.75	9.8
		9.82	9.63	2.48	2.45	12.4	12.4	26.5	10.4
7 Yield round trip	µs	15.2	14.9	3.69	3.67	8.78	8.74	28.6	28.8
		19.5	19.5	4.78	4.85	18.1	18.1	29.1	28.8
8 Preemption latency	µs	5.23	5.3	1.38	1.38	2.79	2.89	9.12	9.47
		9.3	9.37	2.39	2.37	12.2	12.3	9.73	10.2
9 Priority inheritance	µs	12.5	12.5	3.13	3.05	7.03	7.19	23.2	23.7
		16.5	16.6	3.2	3.14	16.4	16.6	23.2	41.1
10 Scheduler overhead	µs	0.5	0.524	0.156	0.163	0.419	0.531	1.5	1.83
		0.833	0.893	0.158	0.171	9.78	9.89	1.5	1.83
11 Task notification	µs	5.22	5.11	1.21	1.19	2.76	2.84	9.06	9.14
		9.29	9.18	2.25	2.23	12.1	12.2	9.64	9.62
IPC									
12 IPC ping-pong	µs	24.8	24.8	6.65	6.41	24.4	24.6	47.9	48.9
		29.2	30.7	7.83	7.49	42	42.3	65.8	66.2
13 Payload sweep	µs	63	63.1	10.8	10.6	29.7	29.9	177	178
		64.9	65.3	11.9	11.9	39.1	47.6	195	196
14 IPC throughput	B/s	3.05M	3.04M	12.5M	13.1M	5.95M	5.88M	1.34M	1.32M
		3.05M	3.04M	12.7M	13.1M	5.95M	5.88M	1.34M	1.32M
15 Zero-copy IPC	ticks	5.53k	5.58k	2.04k	2k	679	677	8.08k	8.05k
		5.55k	5.58k	2.07k	2.03k	679*	677*	8.08k	8.05k

Table IX: NuttX benchmark results by board and language

Benchmark	Unit	STM32F4		STM32H723		ESP32-C6		nRF52840	
		C	Rust	C	Rust	C	Rust	C	Rust
Interrupts									
16 Interrupt handler to task	µs	5.41	5.41	1.15	1.16	2.17	2.17	10.9	11
		9.55	9.55	1.19	2.18	19.9	19.9	11.2	11.6
17 Trigger to interrupt	µs	0.607	0.631	0.244	0.21	4.86	4.87	1.77	1.69
		0.649	0.667	0.25	0.213	4.88	4.96	1.8	1.69
18 Nested interrupt	µs	52.5	52.6	12.7	12.7	37.4	37.3	79.2	79
		55.4	55.5	13.7	13.6	47	46.9	80	79
19 Tick overhead	µs	·	·	·	·	·	·	·	·
20 Observed interrupt deferral	µs	0.75	0.75	0.321	0.337	8.45	8.44	1.88	2.67
		4.71	4.32	0.637	1.2	17.8	17.8	13.9	17
Timing & Sleep									
21 Periodic jitter	µs	0	0	0	0	985	985	0.0156	0
		4.15	4.15	1.03	1.03	985	985	0.0317	0.00109
22 Loaded periodic jitter	µs	5.56	5.55	1.25	1.25	19.4	19.4	12.4	12.8
		57.8	57.7	49*	49	80.6	80.6	77.6	77.4
23 Sleep accuracy	µs	997	997	999	999	978	978	993	994
		997	997	1k	1k	978	978	993	994
Load									
24 Tick-related CPU share while idle	ppm	·	·	·	·	·	·	·	·
25 CPU share under load	ppm	·	·	·	·	·	·	·	·
Memory									
26 Heap fragmentation	ppm	672k	673k	667k	668k	565k	569k	667k	669k
		672k	673k	667k	668k	565k	569k	667k	669k

NuttX benchmark results by board and language (continued)

Benchmark	Unit	STM32F4		STM32H723		ESP32-C6		nRF52840	
		C	Rust	C	Rust	C	Rust	C	Rust
Basic API									
1 Timer and runner overhead	µs	0.565	0.446	0.167	0.146	1.25	1.12	0.375	0.453
		0.565	0.446	0.167	0.146	1.25	1.12	0.375	0.453
2 Queue send and receive	µs	2.51	2.82	0.687	0.615	2.81	2.5	5.38	5.91
		2.51	2.82	0.708	0.615	2.88	2.5	5.38	5.94
3 Semaphore latency	µs	0.875	0.893	0.319	0.327	1.44	1.44	1.23	1.44
		0.875	0.893	0.319	0.327	1.44	1.5	1.23	1.47
4 Mutex uncontended	µs	1.2	1.23	0.382	0.392	1.75	1.81	2.42	2.59
		1.2	1.23	0.385	0.398	1.75	1.81	2.42	2.59
5 Allocation latency	µs	6.65	6.76	2.7	2.69	11.5	11.6	14.5	14.8
		6.65	6.76	2.7	2.69	11.5	11.6	14.7	14.8
Scheduling									
6 Task release to receiver	µs	8.7	8.65	2.14	2.13	8.94	8.88	24.2	23.7
		8.7	8.65	2.14	2.13	456	464	24.2	23.7
7 Yield round trip	µs	27.9	28.6	6.97	7.04	66.1	66.2	98.9	95.8
		27.9	28.6	7.05	7.08	86.2	86.4	98.9	95.8
8 Preemption latency	µs	8.21	8.49	2.07	2.1	8.75	8.75	23.5	23.3
		8.21	8.49	2.08	2.1	469	471	23.6	23.3
9 Priority inheritance	µs	28.2	29	7.29	7.33	57.2	57.4	99.5	98.6
		28.2	29.1	7.3	7.33	70.6	70.9	99.5	98.6
10 Scheduler overhead	µs	3.26	3.21	0.829	0.809	1.56	1.62	6.88	6.92
		3.26	3.21	0.86	0.842	1.62	1.69	6.88	6.92
11 Task notification	µs	10.5	10.8	2.71	2.71	23.2	23.2	34.6	34.2
		10.5	10.8	2.71	2.71	29.9	30	34.6	34.2
IPC									
12 IPC ping-pong	µs	18.8	20.3	4.57	4.77	28.1	28.2	51.9	55.2
		18.8	20.3	4.57	4.77	488	485	52	55.2
13 Payload sweep	µs	45.8	31.5	7.57	6.58	31.3	31.4	111	75.6
		45.8	31.5	7.58	6.59	492	480	111	75.6
14 IPC throughput	B/s	3.45M	3.44M	15.1M	15M	1.34M	1.34M	1.23M	1.27M
		3.48M	3.45M	15.2M	15M	1.34M	1.34M	1.23M*	1.27M*
15 Zero-copy IPC	ticks	4.55k	1.41k	1.52k	690	51	51	3.8k	1.27k
		4.55k	1.41k	1.56k	695	7.1k*	7.23k*	3.8k	1.27k

Table X: Zephyr benchmark results by board and language

Benchmark	Unit	STM32F4		STM32H723		ESP32-C6		nRF52840	
		C	Rust	C	Rust	C	Rust	C	Rust
Interrupts									
16 Interrupt handler to task	µs	9.58	9.66	2.36	2.42	8.25	8.31	26.4	26.2
		9.58	9.66	2.36	2.46	8.31	8.31	26.4	26.2
17 Trigger to interrupt	µs	0.667	0.744	0.202	0.204	1.69	1.75	0.703	0.672
		0.667	0.744	0.204	0.213	1.75	1.75	0.703	0.672
18 Nested interrupt	µs	33.6	33.6	6.81	6.81	33	33	90.4	73.6
		33.6	33.6	6.81	6.81	33	33	90.4	73.6
19 Tick overhead	µs	·	·	·	·	·	·	·	·
20 Observed interrupt deferral	µs	0.708	0.69	0.283	0.298	2.38	2.31	2	2.2
		0.714	0.69	0.287	0.302	2.38	2.44	2.14	2.36
Timing & Sleep									
21 Periodic jitter	µs	0	0	0	0	0	0	916	80.9
		0	0	0	0	0	0	916	80.9
22 Loaded periodic jitter	µs	9.11	9.28	18.8	19.9	43.1	41.5	53.6	53.7
		57.2	57.5	47.6*	47.6*	492	486	85	85
23 Sleep accuracy	µs	997	997	999	1k	997	998	76.5	75.3
		997	997	999	1k	997	998	76.5	75.3
Load									
24 Tick-related CPU share while idle	ppm	·	·	·	·	·	·	·	·
25 CPU share under load	ppm	·	·	·	·	·	·	·	·
Memory									
26 Heap fragmentation	ppm	820k	820k	820k	820k	850k	850k	820k	820k
		820k	820k	820k	820k	850k	850k	820k	820k

Zephyr benchmark results by board and language (continued)

Benchmark	Unit	STM32F4 Rust	STM32H723 Rust
Basic API			
1 Timer and runner overhead	µs	0.00595 0.00595	0.00192 0.00192
2 Queue send and receive	µs	3.67 3.67	2.52 2.54
3 Semaphore latency	µs	2.58 2.58	1.76 1.76
4 Mutex uncontended	µs	3.09 3.09	2 2
5 Allocation latency	µs	3.03 3.03	2 2
Scheduling			
6 Task release to receiver	µs	12.6 13.6	8.88 8.88
7 Yield round trip	µs	5.21 5.21	3.66 3.66
8 Preemption latency	µs	3.08 3.08	2.14 2.14
9 Priority inheritance	µs	13.2 13.2	9.03 9.03
10 Scheduler overhead	µs	1.39 1.39	0.931 0.931
11 Task notification	µs	12.3 13.8	8.6 8.61
IPC			
12 IPC ping-pong	µs	19.4 20.9	13.8 14.7
13 Payload sweep	µs	47.7 50.3	30.1 31.2
14 IPC throughput	B/s	5.3M	7.3M
15 Zero-copy IPC	ticks	798 798	1.78k 1.79k

Table XI: FreeRTOS-in-Rust benchmark results by board and language

Benchmark	Unit	STM32F4 Rust	STM32H723 Rust
Interrupts			
16 Interrupt handler to task	µs	3.17 3.17	2.45 3.56
17 Trigger to interrupt	µs	0.179 0.179	0.125 0.125
18 Nested interrupt	µs	47 47	9.44 9.44
19 Tick overhead	µs	1.4 1.4	0.638 0.65
20 Observed interrupt deferral	µs	1.32 1.37	1.15 1.15
Timing & Sleep			
21 Periodic jitter	µs	0.0893 0.0893	0.0308 0.05
22 Loaded periodic jitter	µs	3.3 57	2.4 49.2
23 Sleep accuracy	µs	0.0952 0.0952	0.0808 1.08
Load			
24 Tick-related CPU share while idle	ppm	635 635	348 348
25 CPU share under load	ppm	1M 1M	1M 1M
Memory			
26 Heap fragmentation	ppm	727k 727k	727k 727k

FreeRTOS-in-Rust benchmark results by board and language (continued)

Benchmark	Unit	STM32F4 Rust
Basic API		
1 Timer and runner overhead	µs	0.399 0.399
2 Queue send and receive	µs	1.49 1.49
3 Semaphore latency	µs	3.7 3.7
4 Mutex uncontended	µs	8.36 8.36
5 Allocation latency	µs	.
Scheduling		
6 Task release to receiver	µs	15.5 15.5
7 Yield round trip	µs	11.6 28.9
8 Preemption latency	µs	1.71 1.71
9 Priority inheritance	µs	1.69 1.69
10 Scheduler overhead	µs	2.26 2.35
11 Task notification	µs	12.2 12.2
IPC		
12 IPC ping-pong	µs	11.6 11.6
13 Payload sweep	µs	62.5 62.6
14 IPC throughput	B/s	6.85M
15 Zero-copy IPC	ticks	8.57k 8.57k

Table XII: Rivet benchmark results by board and language

Benchmark	Unit	STM32F4 Rust
Interrupts		
16 Interrupt handler to task	µs	3.64 3.64
17 Trigger to interrupt	µs	0.643 0.643
18 Nested interrupt	µs	16.8 16.9
19 Tick overhead	µs	5.13 5.13
20 Observed interrupt deferral	µs	1.57 1.64
Timing & Sleep		
21 Periodic jitter	µs	0.661 0.661
22 Loaded periodic jitter	µs	6.86 96.3
23 Sleep accuracy	µs	375 555
Load		
24 Tick-related CPU share while idle	ppm	4.85k 4.85k
25 CPU share under load	ppm	998k 998k
Memory		
26 Heap fragmentation	ppm	·

Rivet benchmark results by board and language (continued)

B.1 Filtered observations

Fractions are the excluded or sentinel proportions of all observations; n is the retained count. They are separated from the dense value tables so no table is clipped.

Case	RTOS	Configuration	Excluded fraction
12 Loaded periodic jitter	FreeRTOS	STM32F4 C	98 % miss, n=4
12 Loaded periodic jitter	FreeRTOS	STM32F4 Rust	98 % miss, n=6
12 Loaded periodic jitter	FreeRTOS	STM32H723 C	48 % miss, n=131
12 Loaded periodic jitter	FreeRTOS	STM32H723 Rust	48 % miss, n=131
12 Loaded periodic jitter	NuttX	STM32H723 C	0.1 % miss, n=999
12 Loaded periodic jitter	Zephyr	STM32H723 C	53 % miss, n=470
12 Loaded periodic jitter	Zephyr	STM32H723 Rust	56 % miss, n=440
20 IPC throughput	Zephyr	nRF52840 C	2 % invalid, n=980
20 IPC throughput	Zephyr	nRF52840 Rust	1 % invalid, n=987
28 Zero-copy IPC	FreeRTOS	ESP32-C6 C	2 % no benefit, n=980
28 Zero-copy IPC	FreeRTOS	ESP32-C6 Rust	2 % no benefit, n=976
28 Zero-copy IPC	NuttX	ESP32-C6 C	0.1 % no benefit, n=999
28 Zero-copy IPC	NuttX	ESP32-C6 Rust	0.6 % no benefit, n=994
28 Zero-copy IPC	Zephyr	ESP32-C6 C	13 % no benefit, n=874
28 Zero-copy IPC	Zephyr	ESP32-C6 Rust	8 % no benefit, n=923

C Complete Memory Results

Table XIII lists the runtime memory collector fields for all 27 firmware configurations, and Table XIV lists their static flash and RAM footprints. A dot or dash marks a field that the adapter does not report. In the runtime table, `Heap over` is allocation overhead, `Task RAM` is the kernel-specific task overhead, `Worst` and `Best` are the minimum and maximum free stack values across tasks, `Bench stack free` is the benchmark task's remaining stack headroom, `Heap min` is the minimum observed free heap, and `Heap free` is free heap at completion. All values are bytes except the task count. The three Rust-native rows are preserved historical summaries because their raw campaign bundle is unavailable.

Runtime memory measurements

bytes unless noted; · or - = not reported

Path	Heap over	Task RAM	Tasks	Worst	Best	Bench stack free	Heap min	Heap free
FreeRTOS								
STM32F4 C	15	112	3	404	14728	14728	16344	83928
STM32F4 Rust	15	112	3	404	15348	15348	16336	83920
STM32H723 C	15	112	3	404	14712	14712	16344	83928
STM32H723 Rust	15	112	3	404	15348	15348	16336	83920
ESP32-C6 C	15	352	4	1248	3868	2304	298364	375100
ESP32-C6 Rust	15	352	4	1248	3868	2340	298508	375244
nRF52840 C	15	112	3	404	14656	14656	16336	83920
nRF52840 Rust	15	112	3	404	15400	15400	16344	83928
NuttX								
STM32F4 C	23	976	2	312	712	312	·	92768
STM32F4 Rust	23	976	2	336	712	336	·	91176
STM32H723 C	23	976	2	704	2328	2328	·	251760
STM32H723 Rust	23	976	2	704	2368	2368	·	250168
ESP32-C6 C	15	952	2	140	1344	140	·	379432
ESP32-C6 Rust	15	952	2	300	1344	300	·	377864
nRF52840 C	23	976	2	312	716	312	·	160144
nRF52840 Rust	23	976	2	336	716	336	·	158536
Zephyr								
STM32F4 C	15	144	2	272	2728	2728	·	1884
STM32F4 Rust	15	144	2	272	3008	3008	·	1372
STM32H723 C	15	144	2	272	2672	2672	·	165636
STM32H723 Rust	15	144	2	272	3000	3000	·	100612
ESP32-C6 C	23	248	2	384	2576	2576	·	301376
ESP32-C6 Rust	15	248	2	384	2896	2896	·	230604
nRF52840 C	15	144	2	272	2728	2728	·	100100
nRF52840 Rust	15	144	2	272	3008	3008	·	35084
Rust-native (historical summaries)								
FreeRTOS-in-Rust F4 Rust	15	120	18	12384	12384	12384	4008	26976
FreeRTOS-in-Rust H723 Rust	15	120	18	12384	12384	12384	4000	26968
Rivet F4 Rust	-	100	11	344	5320	5320	-	-

Table XIII: Runtime memory fields by configuration; matched rows derive from the campaign selected by the thesis evidence lock. Bytes except task count. `Bench stack free` is the benchmark task's minimum unused stack. · or - means not reported. Rust-native rows are preserved historical summaries because their raw campaign bundle is unavailable.

Path	Flash (KiB)	RAM (KiB)
FreeRTOS STM32F4 C	41.5	176.1
FreeRTOS STM32F4 Rust	50.0	175.7
FreeRTOS STM32H723 C	46.1	175.2
FreeRTOS STM32H723 Rust	53.5	174.8
FreeRTOS ESP32-C6 C	195.1	174.4
FreeRTOS ESP32-C6 Rust	203.3	206.3
FreeRTOS nRF52840 C	37.2	173.6
FreeRTOS nRF52840 Rust	45.0	173.2
NuttX STM32F4 C	112.5	96.2
NuttX STM32F4 Rust	130.9	97.8
NuttX STM32H723 C	101.7	99.0
NuttX STM32H723 Rust	119.6	100.6
NuttX ESP32-C6 C	177.0	311.8
NuttX ESP32-C6 Rust	219.8	406.3
NuttX nRF52840 C	113.0	94.5
NuttX nRF52840 Rust	131.4	96.0
Zephyr STM32F4 C	39.1	126.0
Zephyr STM32F4 Rust	51.6	126.5
Zephyr STM32H723 C	41.1	158.1
Zephyr STM32H723 Rust	59.9	221.6
Zephyr ESP32-C6 C	76.2	250.2
Zephyr ESP32-C6 Rust	118.9	319.2
Zephyr nRF52840 C	41.9	158.1
Zephyr nRF52840 Rust	60.6	221.6
FreeRTOS-in-Rust STM32F4 Rust	38.0	78.3
FreeRTOS-in-Rust STM32H723 Rust	39.5	78.3
Rivet RTOS STM32F4 Rust	78.3	46.7

Table XIV: Static footprint by configuration; estimates exclude bootloaders and partition tables.

D Complete Safety Analysis Results

The tables report resolved source operations separately from unresolved records. Unresolved records are excluded from the resolved-operation denominators and are not evidence that an operation is absent. A resolved record identifies a source-level operation; it does not establish that the complete firmware is memory-safe. Trace coverage and profile labels identify the scope of each table.

RTOS	Component	Raw deref.	Pointer arith.	Pointer conv.	Macro ptr. cast	Unsafe context
FreeRTOS	C application	43	16–18	1–3	22–25	·
	C adapter	47–142	2–6	0–4	281–362	·
	Rust application	0	0	0	·	403/404
	Rust FFI boundary	17	17	44	·	241/241
Zephyr	C application	6–13	16–20	1	10	·
	C adapter	25–70	0	3	135–146	·
	Rust application	0	0	0	·	37/52
	Rust FFI boundary	172	0	55	·	557/557
NuttX	C application	6	17	1	9–10	·
	C adapter	54–104	0	27	160–189	·
	Rust application	0	0	0	·	42/58
	Rust FFI boundary	169	0	44	·	627/627

Table XV: Active resolved operations by source-path role. Ranges span boards; C and Rust categories are not directly comparable.

Path	Role	Records	Deref.	Field	Arith.	Cast	Macro cast	Array	Kernel	Global
FreeRTOS										
STM32F4 Discovery	C application	177	43	41	16	3	22	15	1	36
STM32F4 Discovery	C adapter	862	47	0	2	4	281	70	125	322
STM32F4 Discovery	C support	1039	90	41	18	7	303	85	126	358
STM32H723	C application	177	43	41	16	3	22	15	1	36
STM32H723	C adapter	862	47	0	2	4	281	70	125	322
STM32H723	C support	1039	90	41	18	7	303	85	126	358
nRF52840 DK	C application	169	43	41	16	3	22	15	1	28
nRF52840 DK	C adapter	991	142	0	6	0	362	70	111	289
nRF52840 DK	C support	1160	185	41	22	3	384	85	112	317
ESP32-C6	C application	182	43	41	18	1	25	15	1	38
ESP32-C6	C adapter	1204	59	0	2	2	309	68	235	525
ESP32-C6	C support	1484	102	41	20	6	379	79	290	563
Zephyr										
STM32F4 Discovery	C application	151	6	7	16	1	10	26	0	83
STM32F4 Discovery	C adapter	702	25	77	0	3	140	52	12	393
STM32F4 Discovery	C support	1052	35	28	16	4	227	93	79	568
STM32H723	C application	150	6	7	16	1	10	25	0	83
STM32H723	C adapter	698	25	77	0	3	135	52	12	394
STM32H723	C support	1103	35	84	16	4	220	93	79	570
nRF52840 DK	C application	151	6	7	16	1	10	26	0	83
nRF52840 DK	C adapter	747	69	77	0	3	145	52	12	389
nRF52840 DK	C support	1157	84	84	16	4	232	93	79	563
ESP32-C6	C application	163	13	7	20	1	10	27	0	83
ESP32-C6	C adapter	790	70	77	0	3	146	52	12	430
ESP32-C6	C support	1214	93	84	20	4	234	94	79	604
NuttX										
STM32F4 Discovery	C application	170	6	9	17	1	10	32	0	93
STM32F4 Discovery	C adapter	878	79	78	0	27	172	33	11	469
STM32F4 Discovery	C support	1300	91	87	17	37	261	78	80	636
STM32H723	C application	155	6	9	17	1	9	27	0	84
STM32H723	C adapter	723	54	0	0	27	160	33	0	440
STM32H723	C support	1036	63	9	17	34	249	74	6	571
nRF52840 DK	C application	170	6	9	17	1	10	32	0	93
nRF52840 DK	C adapter	898	94	78	0	27	186	33	11	460
nRF52840 DK	C support	1320	106	87	17	37	275	78	80	627
ESP32-C6	C application	170	6	9	17	1	10	32	0	93
ESP32-C6	C adapter	966	104	78	0	27	189	33	11	515
ESP32-C6	C support	1388	116	87	17	37	278	78	80	682

Table XVI: C operation categories by role and configuration

Path	C app.	C adapter	C support	Shared	Rust app.	Rust FFI
FreeRTOS						
STM32F4 Discovery	177	862	1039	10	404	241
STM32H723	177	862	1039	10	404	241
nRF52840 DK	169	991	1160	10	404	241
ESP32-C6	182	1204	1484	367	404	241
Zephyr						
STM32F4 Discovery	151	702	1052	367	52	557
STM32H723	150	698	1103	367	52	557
nRF52840 DK	151	747	1157	367	52	557
ESP32-C6	163	790	1214	367	52	557
NuttX						
STM32F4 Discovery	170	878	1300	367	58	627
STM32H723	155	723	1036	10	58	627
nRF52840 DK	170	898	1320	367	58	627
ESP32-C6	170	966	1388	367	58	627
Rust-native						
FreeRTOS-in-Rust / STM32F4 Discovery	·	·	·	2	179	0
FreeRTOS-in-Rust / STM32H723	·	·	·	2	179	0
Rivet / STM32F4 Discovery	·	·	·	2	8	0

Table XVII: Operation counts by source role and configuration

Path	Role	Records	Raw deref.	Unres. raw	Arith.	Cast	Kernel	Global	Unsafe
FreeRTOS									
STM32F4 Discovery	Rust application	404	0	0	0	0	186	216	403/404
STM32F4 Discovery	Rust FFI boundary	241	17	0	17	44	66	93	241/241
STM32H723	Rust application	404	0	0	0	0	186	216	403/404
STM32H723	Rust FFI boundary	241	17	0	17	44	66	93	241/241
nRF52840 DK	Rust application	404	0	0	0	0	186	216	403/404
nRF52840 DK	Rust FFI boundary	241	17	0	17	44	66	93	241/241
ESP32-C6	Rust application	404	0	0	0	0	186	216	403/404
ESP32-C6	Rust FFI boundary	241	17	0	17	44	66	93	241/241
Zephyr									
STM32F4 Discovery	Rust application	52	0	0	0	0	52	0	37/52
STM32F4 Discovery	Rust FFI boundary	557	172	29	0	55	81	249	557/557
STM32H723	Rust application	52	0	0	0	0	52	0	37/52
STM32H723	Rust FFI boundary	557	172	29	0	55	81	249	557/557
nRF52840 DK	Rust application	52	0	0	0	0	52	0	37/52
nRF52840 DK	Rust FFI boundary	557	172	29	0	55	81	249	557/557
ESP32-C6	Rust application	52	0	0	0	0	52	0	37/52
ESP32-C6	Rust FFI boundary	557	172	29	0	55	81	249	557/557
NuttX									
STM32F4 Discovery	Rust application	58	0	1	0	0	56	0	42/58
STM32F4 Discovery	Rust FFI boundary	627	169	19	0	44	159	255	627/627
STM32H723	Rust application	58	0	1	0	0	56	0	42/58
STM32H723	Rust FFI boundary	627	169	19	0	44	159	255	627/627
nRF52840 DK	Rust application	58	0	1	0	0	56	0	42/58
nRF52840 DK	Rust FFI boundary	627	169	19	0	44	159	255	627/627
ESP32-C6	Rust application	58	0	1	0	0	56	0	42/58
ESP32-C6	Rust FFI boundary	627	169	19	0	44	159	255	627/627

Table XVIII: Rust operation categories by role and configuration

Path	Cases	Comparable	C call sites	Rust call sites	FFI symbols	Unsafe calls
freertos-esp32c6-all	22	21	431	296	75	213
freertos-nrf52840-all	24	23	439	326	75	237
freertos-stm32f4-all	26	25	447	343	75	250
freertos-stm32h723-all	26	25	447	343	75	250
nuttx-esp32c6-all	23	22	562	408	73	227
nuttx-nrf52840-all	23	22	548	408	73	227
nuttx-stm32f4-all	23	22	553	408	73	227
nuttx-stm32h723-all	23	22	553	408	73	227
zephyr-esp32c6-all	23	22	380	364	63	190
zephyr-nrf52840-all	23	22	418	364	63	190
zephyr-stm32f4-all	23	22	416	364	63	190
zephyr-stm32h723-all	23	22	428	364	63	190

Table XIX: Source-tracing coverage and call-site volume

Configuration	Cases	Complete	Boundary	Records	Raw deref.	Unres. raw	Cast	Kernel	Unsafe
FreeRTOS-in-Rust / STM32F4 Discovery	26	5	21	181	1	21	0	30	176/181
FreeRTOS-in-Rust / STM32H723	26	5	21	181	1	21	0	30	176/181
Rivet / STM32F4 Discovery	24	1	23	10	1	31	1	6	4/10

Table XX: Rust-native operation results

E Complete Integration and Complexity Results

Integration surface

project-owned adapter source and configuration counts

System	Adapter C	Adapter Rust	cfg branches	Patch keyword matches
freertos	14109	1893	181	0
freertos-in-rust	0	1771	28	0
nuttx	5920	2050	96	5
rivet	0	1763	2	0
zephyr	12138	2082	78	2

Table XXI: Integration surface by system. Adapter lines are project-owned code; patch counts are keyword matches, not applied patches.

Integration-surface counts cover project-owned adapter sources and plugin code, excluding vendored kernels, generated build trees and retained example applications. They are a heuristic inventory and are not directly comparable to the module-complexity results, which use a different per-module source scope. The build-file inventory is not published because the thesis does not define a verified population for it.

Runner identifier overlap

Jaccard similarity between hand-maintained runners

Pair	C idents	Rust idents	Shared	Similarity
runner.c ↔ lib.rs	194	189	104	0.37

Table XXII: Runner identifier overlap (Jaccard similarity)

Source complexity by module

LOC, function counts, cyclomatic and cognitive complexity, coupling

Module	Lang	LOC	Funcs	cyc med	cyc mean	cyc max	cog med	cog mean	cog max	deps
Shared runner	C	493	4	23	75.2	250	24	92.2	315	2
Shared runner	Rust	636	83	1	3.5	113	0	1.5	61	1
FreeRTOS port	C	2833	238	2	3.2	29	0	2.1	28	12
FreeRTOS app	Rust	1237	94	3	4.2	33	1	2.3	14	5
FreeRTOS FFI	Rust	663	45	2	2.3	6	1	1.3	7	2
NuttX port	C	2627	218	1	2.9	22	0	1.8	35	11
NuttX app	Rust	724	137	1	2.7	31	0	0.5	13	4
NuttX FFI	Rust	1425	103	2	2.7	12	1	1.2	6	2
Zephyr port	C	2428	247	1	2.2	19	0	1.1	21	4
Zephyr app	Rust	685	134	1	2.6	31	0	0.4	18	4
Zephyr FFI	Rust	1454	110	2	2.2	9	0	0.9	5	2

Table XXIII: Source complexity and coupling by module